System design for platform and SRE

Design systems that fail small: estimation, caching, queues, consistency, quorums, cells, shuffle sharding, load shedding, multi-region, change safety, and worked designs reviewed against real outages.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The design method and back-of-envelope estimation (14 min)
2. Caching without stampedes (14 min)
3. Queues, retries, timeouts and idempotency (15 min)
4. Consistency and availability in practice (15 min)
5. Quorums, leaders and split brain (15 min)
6. Clocks, time and ordering (13 min)
7. Partitioning, sharding and consistent hashing (15 min)
8. Storage engines and replication topologies (14 min)
9. Backup, restore and object storage design (14 min)
10. Failure domains, cells and bulkheads (15 min)
11. Blast radius and shuffle sharding (13 min)
12. Graceful degradation and circuit breakers (14 min)
13. Rate limiting, load shedding and admission control (15 min)
14. Multi-region design and global traffic (16 min)
15. Configuration and control plane change safety (15 min)
16. Hidden dependencies and single points of failure (13 min)
17. Observability by design (14 min)
18. Security and identity by design (15 min)
19. Capacity planning and cost trade-offs (14 min)
20. Worked design: a global API with a 99.99 percent SLO (18 min)
21. Worked design: a status page that survives the outage, and an outage design review (16 min)
22. Worked design: a URL shortener (16 min)
23. Worked design: a rate limiter service (16 min)
24. Worked design: a notification fan-out system (16 min)

25. Worked design: a metrics and log pipeline (16 min)
26. Worked design: a multi-tenant Kubernetes platform (17 min)
27. Worked design: an authoritative DNS service (16 min)
28. Worked design: a chat and messaging service (18 min)
29. Worked design: a news feed and timeline (18 min)
30. Worked design: video upload and streaming (18 min)
31. Worked design: a distributed key-value store (19 min)
32. Worked design: a payments ledger (19 min)
33. Worked design: search autocomplete (17 min)
Cheat sheet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 1. The design method and back-of-envelope estimation

14 minutes. A repeatable order for any design question: clarify, estimate, sketch, deep-dive, break it, then state the trade-offs.

System design is not drawing boxes. It is choosing which failure you can afford, and showing your reasoning. Strong designers follow the same order every time, so they never skip the step that would have exposed the flaw. This lesson gives that order and the arithmetic you need to size a system before you draw it.

What you will learn

- Follow six steps: clarify, estimate, sketch, deep-dive, failure modes, trade-offs
- Turn users and behaviour into requests per second, storage and bandwidth
- Pick the numbers that decide the design, and ignore the rest
- Write down assumptions so a reviewer can challenge them

The six steps

1. **Clarify.** Who uses it, what must never be wrong, what may be slow, and what is the SLO? A design for a payment ledger and a design for a like counter are different even if the diagrams match.
2. **Estimate.** Requests per second, data size, growth, read to write ratio.
3. **Sketch.** The smallest design that meets the numbers: clients, entry point, service, store.
4. **Deep-dive.** Pick the one or two parts that carry the risk (the data store, the queue) and design them properly.
5. **Failure modes.** For every box ask what happens when it is slow, down, wrong or overloaded.
6. **Trade-offs.** Say what you gave up. Every design gives something up; naming it is the skill.

Estimation you can do in your head

Round hard. A day is about 100,000 seconds (86,400). So 1 million requests per day is about 10 per second. 100 million per day is about 1,000 per second.

Quantity	Rule of thumb
Seconds per day	86,400, call it 10^5
Peak to average	2x to 10x; use 3x unless you know better
One modern server	thousands of simple requests per second, hundreds for heavy ones
Memory read	around 100 nanoseconds
SSD random read	around 100 microseconds
Same-region network round trip	around 0.5 to 1 millisecond
Cross-continent round trip	100 to 200 milliseconds

Worked example: a service for 20 million daily users, each making 15 requests a day. That is 300 million requests a day, about 3,500 per second on average and about 10,000 per second at a 3x peak. Each response is 5 KB, so peak egress is about 50 MB per second, around 400 Mbit per second. Storage: if each user writes 1 KB a day, that is 20 GB a day, about 7 TB a year before replication. Three copies makes about 22 TB.

Those few lines already decide things: 10,000 per second needs several instances behind a balancer, not one; 22 TB a year does not fit one database node; the read to write ratio decides whether a cache pays.

MARGA by EknathaLabs marga.eknathalabs.com

Which numbers decide the design

Not every figure matters. Look for the number that crosses a threshold: one node's capacity, one network link, one partition's write limit. The design changes where a limit is crossed. If peak is 10,000 per second and one database handles 20,000 writes per second, you do not need sharding yet. Say so, and note at what growth you will.

Write assumptions down

"Assumed 3x peak, 5 KB responses, 99.9 percent of reads can be 60 seconds stale." Each assumption is a place where the design can be wrong. Reviewers can then attack the assumption instead of guessing at it. Several real outages began as an assumption nobody wrote down. Pokemon GO launched in 2016 to 50 times the planned traffic and ten times the worst case estimate; the lesson is not that estimates fail, but that the design needs headroom and a way to scale when they do.

How it fails

Designs fail at the step skipped. Skip clarify and you build the wrong thing. Skip estimate and you build for the wrong size, either one node that melts or a sharded cluster nobody needs. Skip failure modes and the first outage writes them for you.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How The design method and back-of-envelope estimation fits together

Part	What it does	What goes wrong
The six steps	Clarify. Who uses it, what must never be wrong, what may be slow, and what is the SLO?	Never saying what the design gives up.
Estimation you can do in your head	Round hard. A day is about 100,000 seconds (86,400).	Designing for the average instead of the peak.
Which numbers decide the design	Not every figure matters. Look for the number that crosses a threshold: one node's capacity, one network link, one partition's write limit.	Drawing before asking what the SLO is.
Write assumptions down	"Assumed 3x peak, 5 KB responses, 99.9 percent of reads can be 60 seconds stale." Each assumption is a place where the design can be wrong.	Precise arithmetic on guessed inputs (use round numbers).

Flow: The six steps to Estimation you can do in your head / Estimation you can do in your head to Which numbers decide the design / Which numbers decide the design to Write assumptions down

Incident: The launch that was fifty times the plan

Incident: *A game launched worldwide and traffic reached 50 times the initial target within days.*

```
$ kubectl top nodes | sort -k3 -nr | head -3
gke-game-pool-7   7890m   98%   26Gi   91%
gke-game-pool-2   7810m   97%   25Gi   90%
gke-game-pool-9   7760m   97%   25Gi   88%
```

The output above is a simulated illustration. In the real launch, Niantic and Google Cloud added tens of thousands of cores to the Kubernetes backend during the first days. What saved them was a design that could scale horizontally and a vendor who could provide capacity; the estimate itself was wrong by a wide margin. Read the case file for the sourced timeline.

MARGA by EknathaLabs marga.eknathalabs.com

Interview question

"Design a URL shortener for 100 million new links a month." Start by asking about read traffic, link lifetime and custom aliases. Estimate: 100 million a month is about 40 writes per second; reads are perhaps 100 times that, 4,000 per second, so caching matters far more than the write path.

Common mistakes

- Drawing before asking what the SLO is
- Precise arithmetic on guessed inputs (use round numbers)
- Designing for the average instead of the peak
- Never saying what the design gives up

Check yourself

About how many seconds are in a day, for estimation?

Answer: About 100,000 (86,400)

100 million requests per day is roughly how many per second on average?

Answer: About 1,000

What do you do after the sketch and before listing trade-offs?

Answer: Deep-dive the risky parts, then walk the failure modes

Why write assumptions down?

Answer: So reviewers can challenge the exact input that could be wrong

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Caching without stampedes

14 minutes. Where caches help, how invalidation and TTLs go wrong, and how to stop a cold cache from crushing the database behind it.

A cache trades freshness for speed and load. It is the cheapest way to make a read-heavy system fast, and the commonest way to make a recovery fail: when a cache goes cold, every request lands on the store it was protecting. Design the cache and the cold-cache case together.

What you will learn

- Choose a caching pattern: cache-aside, read-through, write-through, write-back
- Set TTLs and invalidate without serving wrong data for long
- Recognise and prevent stampedes, hot keys and cold starts
- Decide when not to cache

Patterns

Cache-aside: the app reads the cache; on a miss it reads the database and fills the cache. Simple and common. **Read-through:** the cache itself loads on a miss. **Write-through:** writes go to cache and store together; reads are fresh, writes are slower. **Write-back:** writes go to the cache and flush later; fast, but data can be lost if the cache dies before flushing. Use write-back only when losing recent writes is acceptable.

TTL and invalidation

A TTL bounds how stale data can be. It is also a promise: "a user may see data up to this old". Pick it from the business rule, not from habit. Explicit invalidation (delete the key when the source changes) is fresher but fails in subtle ways: a delete that is lost leaves stale data until the TTL, so keep a TTL as a safety net even with invalidation.

A classic race: reader A misses and reads old data from the database; writer updates the database and deletes the key; reader A then fills the cache with the old data. The cache is now stale until the TTL. Short TTLs on mutable data, or versioned keys, limit this.

The stampede

A popular key expires. Within milliseconds, thousands of requests miss together and all query the database for the same row. The database slows, so the refill takes longer, so more requests pile in. Defences:

- **Request coalescing:** one request refills, the others wait for it.
- **Jittered TTLs:** add random spread so keys do not expire together.
- **Serve stale while refreshing:** return the old value and refresh in the background.
- **Early refresh:** refresh a little before expiry for hot keys.

The cold start

After a deploy, a cache restart or a failover, the cache is empty. If the database can only handle 10 percent of full traffic, a cold cache at full traffic is an outage. Plan for it: warm the cache before shifting traffic, ramp traffic slowly, and size the database for the miss rate you can reach, not only the steady state.

Hot keys and negative caching

One key getting a large share of traffic overloads the single cache node that holds it. Replicate hot keys or keep a short local in-process cache in front. Also cache "not found" answers briefly, or a missing item will hit the database every time.

MARGA by EknathaLabs
marga.eknathalabs.com

When not to cache

Skip a cache when data must be exactly current (account balances shown for payment), when the hit rate would be low, or when the store is fast enough. A cache you cannot explain is another failure point.

How it fails

Silently. Stale data appears wrong only to the user who saw it. A stampede shows as a database spike right after a cache event. Alert on hit rate, miss rate and database load together; a hit rate that drops while database load rises is the signal.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Caching without stampedes fits together

Part	What it does	What goes wrong
Patterns	Cache-aside: the app reads the cache; on a miss it reads the database and fills the cache.	A database sized only for the warm-cache case.
TTL and invalidation	A TTL bounds how stale data can be. It is also a promise: "a user may see data up to this old".	No jitter on TTLs.
The stampede	A popular key expires. Within milliseconds, thousands of requests miss together and all query the database for the same row.	- Jittered TTLs: add random spread so keys do not expire together.
The cold start	After a deploy, a cache restart or a failover, the cache is empty.	No cache hit-rate alert.
Hot keys and negative caching	One key getting a large share of traffic overloads the single cache node that holds it.	Check the Common mistakes list below this diagram for what usually goes wrong here.
When not to cache	Skip a cache when data must be exactly current (account balances shown for payment), when the hit rate would be low, or when the store is fast enough.	Using write-back for data you cannot lose.

Flow: Patterns to TTL and invalidation / TTL and invalidation to The stampede / The stampede to The cold start / The cold start to Hot keys and negative caching / Hot keys and negative caching to When not to cache

Incident: Recovery that needed a warm cache

Incident: *After a restart the cache was empty and the database fell over under the full read load.*

```
$ redis-cli info stats | grep keyspace
keyspace_hits:0
keyspace_misses:482113
$ psql -c "select count(*) from pg_stat_activity where state='active'"
 count
-------
   398
```

This output is a simulated illustration of a common pattern, not one named outage. The database was sized for a 95 percent hit rate, so after the cache went empty it saw twenty times its normal read load. The team added request coalescing, warmed the cache from a replica before cutover, and ramped traffic in steps.

Interview question

"Your product page is cached for five minutes and a price change must show within seconds. What do you do?" Invalidate on write, keep the TTL as a backstop, and consider versioned keys so a late fill cannot overwrite a newer value.

Common mistakes

- No jitter on TTLs

MARGA by EknathaLabs marga.eknathalabs.com

- A database sized only for the warm-cache case
- Using write-back for data you cannot lose
- No cache hit-rate alert

Check yourself

What is a cache stampede?

Answer: Many requests miss at once and all hit the store for the same data

Which defence spreads out expiry?

Answer: Adding random jitter to TTLs

Why keep a TTL even when you invalidate on write?

Answer: A lost or raced delete would otherwise leave stale data forever

What is the risk of write-back caching?

Answer: Recent writes are lost if the cache fails before flushing

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Queues, retries, timeouts and idempotency

15 minutes. Decouple with queues, retry without causing storms, and make operations safe to repeat.

Networks lose messages and services time out. The question is never whether a call will be repeated, but what happens when it is. Queues absorb bursts, retries recover from blips, and idempotency makes repeats harmless. Done carelessly, the same tools turn a small fault into a storm.

What you will learn

- Choose between a queue and a stream, and what ordering and delivery guarantees mean
- Set timeouts, retries, backoff and jitter so retries help instead of hurt
- Make operations idempotent with keys
- Apply backpressure and dead-letter queues

Queues and streams

A **queue** hands each message to one consumer and removes it after acknowledgement; good for work to be done once. A **stream** is an ordered log that many consumers read at their own pace and can replay; good for events and pipelines. Ordering is usually guaranteed only within a partition, so choose the partition key so related events stay together (for example the account id).

Delivery guarantees: at-most-once (may lose), at-least-once (may repeat), exactly-once (in practice, at-least-once delivery plus idempotent handling). Assume at-least-once and design consumers to be safe on repeats.

Timeouts

Every call needs a timeout shorter than the caller's own timeout. Without one, a slow dependency ties up threads and connections until the caller fails too. Set timeouts from the dependency's real latency (for example just above its 99th percentile), and use a deadline that passes down the whole call chain so work is abandoned once the user has gone.

Retries

Retry only errors that may succeed later (timeouts, 503, connection reset), not ones that never will (400, 404). Use exponential backoff with **jitter**: wait 100 ms, then up to 200, then up to 400, each randomised. Cap the attempts. Without jitter, thousands of clients retry at the same moments and hit the recovering service in waves.

Retry amplification: if each of three layers retries three times, one user request can become 27 calls to the bottom service during an incident, exactly when it is least able to cope. Retry at one layer, and use a **retry budget** (for example, retries may be at most 10 percent of requests).

Idempotency

An operation is idempotent if doing it twice has the same effect as once. Setting a value is; adding to a balance is not. Make non-idempotent operations safe with an **idempotency key**: the client sends a unique key, the server records the key with the result, and a repeat returns the stored result instead of acting again. Payment APIs use this so a retried charge does not bill twice. Store the key and the effect in the same transaction.

Backpressure and dead letters

If producers outrun consumers, queues grow without limit and latency climbs. Apply backpressure: bound the queue, reject or slow producers when it is full, and alert on queue age, not only length. Messages that fail repeatedly go to a **dead-letter queue** so one poison message does not block the rest; someone must watch it.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

Unbounded queues hide overload until memory is gone. Missing jitter creates retry waves. Layered retries multiply load. Consumers without idempotency double-apply work after a redelivery.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Queues, retries, timeouts and idempotency fits together

Part	What it does	What goes wrong
Queues and streams	A queue hands each message to one consumer and removes it after acknowledgement; good for work to be done once.	Assuming exactly-once delivery.
Timeouts	Every call needs a timeout shorter than the caller's own timeout.	Retrying at every layer.
Retries	Retry only errors that may succeed later (timeouts, 503, connection reset), not ones that never will (400, 404).	Retries with no jitter or cap.
Idempotency	An operation is idempotent if doing it twice has the same effect as once.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Backpressure and dead letters	If producers outrun consumers, queues grow without limit and latency climbs.	Alerting on queue length but not age.

Flow: Queues and streams to Timeouts / Timeouts to Retries / Retries to Idempotency / Idempotency to Backpressure and dead letters

Incident: When retries became the outage

> **Incident:** *A dependency slowed down and the clients that called it retried aggressively, keeping it overloaded long after the original fault.*

```
$ grep -c "retry attempt" /var/log/api/app.log
48211
$ grep -c "request received" /var/log/api/app.log
15870
```

This is a simulated illustration of the retry-storm pattern. About three attempts per request, summed across layers, kept the dependency saturated. No single public case is claimed here; the case files on overload and recovery show related effects. The fix was a retry budget, jittered backoff and retries at only one layer.

Interview question

"A queue consumer is slow and the backlog is growing. What do you check, and what do you change?" Check consumer errors and retries, per-message time, partition skew and downstream latency; change by scaling consumers only if the downstream can take it, adding backpressure, and moving poison messages to a dead-letter queue.

Common mistakes

- Retrying at every layer
- Retries with no jitter or cap
- Assuming exactly-once delivery
- Alerting on queue length but not age

Check yourself

Why add jitter to retry backoff?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: So clients do not retry in synchronised waves

What does an idempotency key let a server do?

Answer: Return the stored result for a repeated request instead of acting again

Three layers each retry three times. Worst case calls to the bottom service per user request?

Answer: 27

What belongs in a dead-letter queue?

Answer: Messages that keep failing, set aside so others proceed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Consistency and availability in practice

15 minutes. What CAP and PACELC mean for real choices, and how to pick a consistency level per piece of data.

When a network splits or a replica lags, a system must choose: answer with possibly stale data, or refuse. CAP names that choice. In daily work the sharper question is the trade between consistency and latency when nothing is broken. This lesson turns the theory into per-feature decisions.

What you will learn

- State CAP and PACELC without the myths
- Describe strong, eventual, read-your-writes and causal consistency
- Choose a level per feature, not per system
- Spot replication lag as a source of user-visible bugs

CAP, stated carefully

During a network **partition**, a system cannot both stay **consistent** (every read sees the latest write) and stay **available** (every request gets a non-error answer). Partitions happen, so the real choice is what to give up while one is in progress. It is not a menu of any two of three at all times.

PACELC adds the everyday case: if there is a Partition, choose Availability or Consistency; Else, choose Latency or Consistency. Even with a healthy network, waiting for replicas to confirm a write costs time. Most systems pay that cost somewhere.

Levels you will meet

- **Strong (linearizable):** reads see the latest completed write. Costs latency and availability.
- **Eventual:** replicas converge given time; reads may be stale. Cheap and fast.
- **Read-your-writes:** you see your own writes, others may lag. Fixes the most visible user bug.
- **Monotonic reads:** you never see time go backwards between reads.
- **Causal:** if B depends on A, everyone sees A first.

Choose per feature

A shopping site can use different levels at once. The cart: read-your-writes is enough. Stock count at checkout: strong, or reserve with a check at commit. The product description: eventual, cache freely. A bank balance after a transfer: strong. Ask for each piece of data: what is the cost of a wrong or stale answer, and what is the cost of an error or delay?

Replication lag

Read replicas lag. A user saves a profile, the page reloads from a replica, and the old profile shows. They save again, worse. Fixes: read from the primary for a short time after a write, route a user's reads to the same replica, or carry a version or timestamp token and wait until the replica has reached it. Measure lag and alert on it; lag also decides how much data a failover can lose.

Failover and what it sacrifices

Asynchronous replication keeps writes fast but a primary failure can lose the last writes. Synchronous replication avoids that loss but halts writes if the replica is unreachable. Many teams use one synchronous replica as the failover target. The GoCardless case in 2017 shows a design that had this intent but whose automatic promotion did not trigger, so the outage lasted until engineers promoted a node by hand.

MARGA by EknathaLabs
marga.eknathalabs.com

How it fails

Eventual consistency fails as confusing, not as down: duplicated actions, lost updates when two writers race, double spends. Strong consistency fails as unavailable: writes block during a partition. Name which failure the feature can tolerate.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Consistency and availability in practice fits together

Part	What it does	What goes wrong
CAP, stated carefully	During a network partition, a system cannot both stay consistent (every read sees the latest write) and stay available (every request gets a non-error answer).	Reading from replicas right after a write with no lag handling.
Levels you will meet	Strong (linearizable): reads see the latest completed write.	Never testing failover.
Choose per feature	A shopping site can use different levels at once.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Replication lag	Read replicas lag. A user saves a profile, the page reloads from a replica, and the old profile shows.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Failover and what it sacrifices	Asynchronous replication keeps writes fast but a primary failure can lose the last writes.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: CAP, stated carefully to Levels you will meet / Levels you will meet to Choose per feature / Choose per feature to Replication lag / Replication lag to Failover and what it sacrifices

Incident: The failover that did not fail over

> **Incident:** *The primary database lost its disk array. The synchronous replica was healthy, but the cluster manager did not promote it.*

```
$ crm status | head -5
Online: [ db-a db-b ]
db-a  (ocf::heartbeat:pgsql)  FAILED
db-b  (ocf::heartbeat:pgsql)  Started (not promoted)
$ crm resource promote ms_pgsql
```

This block is a simulated illustration of the pattern. In the real GoCardless incident on 10 October 2017, the API and dashboard were down about 1 hour 50 minutes until a manual promotion. The lesson for design: automatic failover is a feature to be tested on a schedule, not trusted because it exists. See the case file for sourced detail.

Interview question

"Users see their old data right after saving. Walk me through it." Replication lag or a cache; prove which by reading from the primary and checking cache keys; then pick read-your-writes.

Common mistakes

- Choosing one consistency level for the whole system
- Reading from replicas right after a write with no lag handling
- Never testing failover
- Treating CAP as three independent switches

Check yourself

During a partition, CAP says a system must choose between what?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Consistency and availability

What does PACELC add?

Answer: The latency versus consistency trade when there is no partition

Which level fixes a user not seeing their own just-saved change?

Answer: Read-your-writes

Why alert on replication lag?

Answer: It shows stale reads now and the data you could lose on failover

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Quorums, leaders and split brain

15 minutes. How groups of machines agree, why odd numbers matter, and how fencing stops two leaders from corrupting data.

Some jobs must have exactly one owner at a time: the database primary, the holder of a lock, the node that assigns work. Getting machines to agree on who that is, when messages are delayed and nodes pause, is the core problem of distributed systems. You will rarely implement consensus, but you must know how to use and operate it.

What you will learn

- Explain quorum, majority and why clusters use odd sizes
- Describe leader election at the level of Raft style systems
- Define split brain and use fencing to prevent damage
- Know what happens when a consensus store is unhealthy

Quorum

A **quorum** is a majority: more than half. If writes need acknowledgement from a majority, any two majorities share at least one node, so a later read of a majority sees the latest write. With 3 nodes the quorum is 2 and one failure is tolerated. With 5 the quorum is 3 and two are tolerated. 4 nodes tolerate one failure, the same as 3, so use odd sizes. More nodes add fault tolerance but also latency, since more must acknowledge.

Leader election

Consensus systems such as etcd, ZooKeeper and Consul (using Raft or similar protocols) elect one leader. All writes go through it and are replicated to followers; a write is committed once a majority has it. If the leader stops sending heartbeats, followers wait a randomised timeout, one becomes a candidate, and wins with a majority of votes. Each leader period has a number called a term or epoch; messages from an old term are rejected.

Consequence: without a majority, the cluster cannot commit writes. A 3-node cluster that loses 2 nodes stops writing, by design. It chooses consistency over availability.

Split brain

Split brain is two nodes both acting as leader. It happens when a network partition, or a long pause, makes the old leader unaware it was replaced. A frozen process (garbage collection, a VM pause) wakes up and still believes it holds the lock. Two leaders write to the same data and corrupt it.

Fencing

The remedy is **fencing**: the resource itself refuses stale leaders. Give every leadership a monotonically increasing **fencing token**; the storage layer rejects any write whose token is lower than one it has already seen. Other forms: power off the old node (STONITH), revoke its network access, or use leases that expire and a store that checks them. A lease alone is not safe, because a paused process can outlive its lease; the token check is.

Operating consensus stores

Treat etcd, ZooKeeper or Consul as critical infrastructure: run 3 or 5 members across failure domains, keep disks fast (they sync to disk on every commit), monitor leader changes and commit latency, take and test snapshots, and keep their load small. A flood of requests to the consensus store can make the whole platform stall. The 2021 Roblox outage involved a Consul cluster under heavy load and a streaming feature; see the case file for the sourced account.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

Two leaders after a pause. A quorum lost because all members shared one zone. Slow disks causing constant re-elections. A cluster whose members were replaced faster than they could catch up.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Quorums, leaders and split brain fits together

Part	What it does	What goes wrong
Quorum	A quorum is a majority: more than half. If writes need acknowledgement from a majority, any two majorities share at least one node, so a later read of a majority sees...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Leader election	Consensus systems such as etcd, ZooKeeper and Consul (using Raft or similar protocols) elect one leader.	Even-sized or single-zone consensus clusters.
Split brain	Split brain is two nodes both acting as leader.	Trusting a lock lease without a fencing token.
Fencing	The remedy is fencing: the resource itself refuses stale leaders.	Putting heavy application traffic on the consensus store.
Operating consensus stores	Treat etcd, ZooKeeper or Consul as critical infrastructure: run 3 or 5 members across failure domains, keep disks fast (they sync to disk on every commit), monitor...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Quorum to Leader election / Leader election to Split brain / Split brain to Fencing / Fencing to Operating consensus stores

Incident: The leader that did not know

Incident: *A paused primary resumed after failover and kept writing, while the new primary also accepted writes.*

```
$ psql -h old-primary -c "select pg_is_in_recovery()"
 pg_is_in_recovery
-------------------
 f
$ psql -h new-primary -c "select pg_is_in_recovery()"
 pg_is_in_recovery
-------------------
 f
```

This is a simulated illustration of split brain, not one named outage. Both nodes report they are writable. The team had no fencing: the load balancer still sent some traffic to the old primary. They added a fencing step that cuts the old node off before promotion, and a check that only one writable primary exists.

Interview question

"How do you stop two instances of a job both running?" Use a lease from a consensus store plus a fencing token checked by the thing the job writes to; a lease alone cannot stop a paused process.

Common mistakes

- Even-sized or single-zone consensus clusters
- Trusting a lock lease without a fencing token
- Putting heavy application traffic on the consensus store
- Never practising a member replacement

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

A 5-node consensus cluster tolerates how many failures?

Answer: 2

What is split brain?

Answer: Two nodes both acting as the leader

What makes fencing tokens work?

Answer: The resource rejects writes carrying a lower token than it has seen

Why use odd cluster sizes?

Answer: An extra even node adds no failure tolerance

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Clocks, time and ordering

13 minutes. Why you cannot trust wall clocks to order events, and what to use instead.

Programs treat time as a simple number. In a distributed system it is not. Clocks drift, jump and disagree, and the same instant is seen differently by two machines. Designs that assume otherwise lose writes, expire locks early and crash on a leap second.

What you will learn

- Distinguish wall-clock time from monotonic time
- Explain clock skew, jumps and leap seconds
- Order events with logical clocks and versions instead of timestamps
- Design timeouts and expiry that survive clock changes

Two kinds of clock

Wall-clock time is the calendar date and time, synchronised by NTP. It can jump forwards or backwards when corrected. **Monotonic time** only moves forward and is meant for measuring durations. Use the monotonic clock for timeouts and elapsed time, and wall-clock only for showing dates and for coarse expiry.

Skew and drift

Two servers can differ by milliseconds or, if sync breaks, by seconds or more. Last-write-wins by timestamp then silently discards the newer write when it comes from the slower clock. Do not use machine timestamps to decide which of two concurrent writes is correct.

Leap seconds and other jumps

At midnight UTC on 1 January 2017, a leap second was inserted. Cloudflare's custom DNS software computed a time difference that became negative, a value the code assumed could not happen, and a call that panics on negative input failed. Customers with CNAME records saw errors until about 06:45 UTC. Other time bugs: expired certificates, date rollovers, daylight saving transitions, and a Microsoft Exchange date check that failed at the start of 2022. The pattern: code that assumes time only moves forward, evenly.

Ordering without wall clocks

- **Logical clocks (Lamport):** a counter that increases with each event and each received message, giving an order consistent with cause and effect.
- **Version vectors:** per-replica counters that detect true concurrency, so a system can keep both versions and ask for a merge.
- **Database sequence or log offset:** let one authority assign order, as a stream partition does.
- **Hybrid logical clocks:** physical time plus a counter, close to real time and still ordered.

Some databases bound clock uncertainty with special hardware and then wait out the uncertainty before committing. You do not need this, but know the idea: if you must use time for ordering, state the error bound and handle it.

Practical rules

- Run time sync everywhere and alert when offset grows.
- Use monotonic clocks for durations and timeouts.
- Never compute with a time difference you assume is positive; clamp or handle it.
- Store UTC; convert for display only.

MARGA by EknathaLabs
marga.eknathalabs.com

- Test with the clock moved: forwards, backwards, a leap second, a daylight saving change, certificate expiry.
- Track every certificate expiry date; they are clocks too.

How it fails

Silently, until the date arrives. A negative duration, an early expiry, a lock that two holders believe they own, a certificate that lapsed because nobody tracked it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Clocks, time and ordering fits together

Part	What it does	What goes wrong
Two kinds of clock	Wall-clock time is the calendar date and time, synchronised by NTP.	Measuring durations with the wall clock.
Skew and drift	Two servers can differ by milliseconds or, if sync breaks, by seconds or more.	Last-write-wins by wall-clock timestamp.
Leap seconds and other jumps	At midnight UTC on 1 January 2017, a leap second was inserted.	Assuming time differences are never negative.
Ordering without wall clocks	Logical clocks (Lamport): a counter that increases with each event and each received message, giving an order consistent with cause and effect.	No alert on clock offset or certificate expiry.
Practical rules	Run time sync everywhere and alert when offset grows.	- Never compute with a time difference you assume is positive; clamp or handle it.

Flow: Two kinds of clock to Skew and drift / Skew and drift to Leap seconds and other jumps / Leap seconds and other jumps to Ordering without wall clocks / Ordering without wall clocks to Practical rules

Incident: A leap second and a negative number

> **Incident:** *At midnight UTC a time difference computed by the DNS software was negative and a function that rejects negative values panicked.*

```
$ dig +short app.example.com CNAME
;; connection timed out; no servers could be reached
$ journalctl -u rrdns --since "00:00" | grep -i panic | head -2
panic: negative duration passed to random selection
```

The console output is a simulated illustration; the real timeline and Cloudflare's own explanation are in the case file. The fix was to handle the negative value, and the lasting lesson is to treat time inputs as untrusted and to test timestamps that move backwards.

Interview question

"How do you make sure only one worker processes a job when clocks differ?" Do not depend on timestamps. Use a lock with a fencing token, or let a queue partition assign each job to a single consumer.

Common mistakes

- Last-write-wins by wall-clock timestamp
- Measuring durations with the wall clock
- Assuming time differences are never negative
- No alert on clock offset or certificate expiry

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which clock should you use to measure a timeout?

Answer: The monotonic clock

Why is last-write-wins by timestamp risky?

Answer: A slow clock can make a newer write look older and lose it

What orders events without wall clocks?

Answer: Logical clocks, version vectors or a single sequencing log

What went wrong in the 2017 Cloudflare leap second case?

Answer: A time difference became negative and the code could not handle it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Partitioning, sharding and consistent hashing

15 minutes. Split data across nodes before one node runs out, choose a partition key, and avoid hot keys and painful reshuffles.

One node always has a ceiling: disk size, write throughput, memory, a file system limit. Partitioning splits data across nodes so each holds a share. It is the main way storage scales, and it is hard to change later, so the choices you make early (the key, the scheme, the number of partitions) matter more than the technology.

What you will learn

- Choose between range, hash and directory partitioning
- Pick a partition key and spot hot keys and skew
- Explain consistent hashing and why it limits data movement
- Plan resharding and cross-partition queries

Why partition

Reasons are size (data no longer fits), write rate (one primary cannot take the writes) and blast radius (a fault should reach only one partition). Replication copies the same data to more nodes for availability and read scale; it does not help writes or size. Partitioning divides data; replication copies it. Real systems do both: each partition has its own replicas.

Schemes

- **Range:** keys A to F on node 1, G to M on node 2. Range scans are cheap, but sequential keys (timestamps, auto-increment ids) pile all new writes on the last range, a hot spot.
- **Hash:** hash the key and take it modulo N, or map it to a range of hash values. Spreads load evenly; range scans must visit every partition.
- **Directory (lookup):** a table says which partition owns each key or tenant. Flexible, and the directory becomes a critical dependency to cache and protect.

Choosing the key

A good key has many distinct values, spreads reads and writes evenly, and keeps data that is used together on the same partition. A tenant id keeps one customer's data together, which suits a cell design, but one very large tenant becomes a hot partition. A user id spreads well. A country code does not (a few huge values). Ask: what is the biggest value of this key, and what fraction of traffic does it carry?

Hot keys and skew

Even with a hash, one key can dominate: a celebrity account, a viral item, a default value used by millions. Remedies: add a random suffix to spread one key over several (and read all of them), cache the hot key in front, split the hot entity into smaller pieces, or give large tenants their own partitions. Monitor per-partition load, not only the average.

Modulo is a trap, consistent hashing is the fix

With `hash(key) mod N`, changing N from 10 to 11 moves about 90 percent of keys to different nodes, a storm of data movement and cache misses. **Consistent hashing** places nodes and keys on a ring; a key belongs to the next node clockwise. Adding or removing one node moves only about 1/N of the keys. To even out load, each physical node takes many positions on the ring (virtual nodes). Many caches and distributed stores use it.

MARGA by EknathaLabs
marga.eknathalabs.com

Another approach is a fixed, large number of logical partitions (for example 1,024) mapped to nodes. Moving a partition moves a whole unit, and the partition count never changes.

Cross-partition work

Queries that span partitions (joins, global secondary indexes, uniqueness checks, transactions) are slow or hard. Design so the common case touches one partition. Use a local index per partition and scatter-gather for rare global queries, accept eventual consistency for global indexes, and use sagas or outbox patterns rather than distributed transactions where you can.

Resharding

Plan it before you need it: choose a scheme that splits without downtime (split a partition in two while it serves traffic), copy data in the background, switch with a version of the directory, verify, then delete. Waiting until the node is full leaves no room to copy.

How it fails

A partition key chosen for convenience that skews. A modulo scheme that makes growth an outage. A directory that is a single point. A rebalance that saturates the network and slows production. A limit nobody watched until the single node hit it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Partitioning, sharding and consistent hashing fits together

Part	What it does	What goes wrong
Why partition	Reasons are size (data no longer fits), write rate (one primary cannot take the writes) and blast radius (a fault should reach only one partition).	Sequential partition keys.
Schemes	Range: keys A to F on node 1, G to M on node 2.	hash mod N with no plan for growth.
Choosing the key	A good key has many distinct values, spreads reads and writes evenly, and keeps data that is used together on the same partition.	Ignoring one huge tenant or key.
Modulo is a trap, consistent	With hash(key) mod N, changing N from 10 to 11 moves about 90 percent of keys to different nodes, a storm of data movement and cache misses.	No alert on distance to a node limit.
Cross-partition work	Queries that span partitions (joins, global secondary indexes, uniqueness checks, transactions) are slow or hard.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Resharding	Plan it before you need it: choose a scheme that splits without downtime (split a partition in two while it serves traffic), copy data in the background, switch with a...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Why partition to Schemes / Schemes to Choosing the key / Choosing the key to Modulo is a trap, consistent / Modulo is a trap, consistent to Cross-partition work / Cross-partition work to Resharding

Incident: A table that hit a size limit

Incident: *A single database table on one node grew until it reached a file size limit, and the service stopped.*

```
$ ls -lh /var/lib/mysql/shop/bookmarks.ibd
-rw-rw---- 1 mysql mysql 2.0T bookmarks.ibd
mysql> insert into bookmarks values (...);
ERROR 1114 (HY000): The table 'bookmarks' is full
```

The console lines are a simulated illustration. In the real Instapaper incident on 9 February 2017, a MySQL table on Amazon RDS reached a 2 TB file size limit; the service was down for about 31 hours and archive access was limited for about five days. The design lesson is to track the distance to every single-node limit, and to partition or archive while there is room to copy. See the case file.

Interview question

"Design the storage for a service that will hold 50 TB of user events." Estimate rate and growth, choose a hashed user id as the key with a large fixed partition count, replicate each partition, handle hot users, and describe how you would add nodes without moving most of the data.

Common mistakes

- Sequential partition keys
- `hash mod N` with no plan for growth
- Ignoring one huge tenant or key
- No alert on distance to a node limit

Check yourself

What does replication give you that partitioning does not?

Answer: Availability and read scale from copies of the same data

Why is hash mod N a problem when N changes?

Answer: Most keys move to different nodes

What does consistent hashing limit?

Answer: The share of keys that move when a node is added or removed

Which key is most likely to create a hot partition?

Answer: An auto-increment id used with range partitioning

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Storage engines and replication topologies

14 minutes. How B-trees and LSM trees differ, what leader-follower, multi-leader and leaderless replication trade, and how data stores fail from the inside.

You do not need to write a database, but you must know what is underneath one, because capacity limits, latency spikes and recovery times come from it. This lesson covers the two main storage engine families and the three ways to replicate, with an eye on how each fails.

What you will learn

- Compare B-tree and LSM tree storage engines
- Explain write-ahead logs, compaction and vacuum, and why they matter in operations
- Choose a replication topology and know its conflict risk
- Spot internal limits such as transaction ids and metadata growth

Two storage engine families

B-tree engines (many relational databases) keep sorted pages on disk and update them in place. Reads are predictable and a point lookup touches a few pages. Writes modify pages, so each write costs random I/O, and a write-ahead log (WAL) records changes first so a crash can be recovered.

LSM tree engines (many key-value and wide-column stores) append writes to a memory table and a log, flush sorted files, and merge them in the background (**compaction**). Writes are fast and sequential. Reads may check several files, helped by Bloom filters, and compaction competes with foreground traffic for disk. A burst of writes can build a compaction backlog and push read latency up.

Rule of thumb: LSM favours write-heavy workloads and has to be watched for compaction debt; B-tree favours read-heavy, update-in-place workloads and has to be watched for index bloat and vacuum.

Background work is part of the design

Both families clean up behind the scenes: vacuum and autovacuum, compaction, checkpointing, log truncation, metadata cleanup. When they fall behind, nothing fails at first, then something fails abruptly. Monitor their lag directly. In PostgreSQL, a table whose old row versions are not vacuumed can eventually exhaust transaction ids and the database stops accepting writes to protect itself. Sentry's database did this in 2015 and a Mandrill shard in 2019, with outages of most of a working day and about 40.5 hours respectively. See the case files.

Replication topologies

- **Leader and followers:** one node takes writes and ships a log to followers. Simple and consistent. Failover needs a promotion and fencing, and asynchronous followers can lose recent writes.
- **Multi-leader:** several nodes accept writes, often one per region, and replicate to each other. Good for write latency and regional independence. The price is **conflicts**: two leaders change the same row. You need a rule (last write wins, merge, or application logic) and you must accept that some writes are altered or lost.
- **Leaderless (quorum):** any replica accepts writes; a write succeeds when W of N replicas acknowledge, a read asks R, and R + W > N gives overlap. High availability, tunable consistency, and repair work (read repair, anti-entropy) to converge.

Synchronous and asynchronous

Synchronous replication waits for a follower before acknowledging: no loss on failover, but latency, and a dead follower can block writes. Asynchronous acknowledges at once: fast, but the last writes can be lost. A common middle path is one synchronous follower plus asynchronous ones.

MARGA by EknathaLabs marga.eknathalabs.com

Metadata and hidden growth

Systems also keep metadata: segment lists, tiered storage indexes, file catalogues. If cleanup is not enabled, it grows quietly. In May 2024 Honeycomb found one Kafka broker reading far more from disk than its peers because tiered storage metadata had grown for years without cleanup; enabling the cleaner cut metadata by over 99 percent and broker start-up fell from over 20 minutes to under one minute. Track metadata size and start-up time as signals.

Choosing

Ask about the access pattern (point lookups, scans, writes per second), the consistency each feature needs, the growth, and the operations your team can actually run. A familiar relational database with replicas and partitions solves more problems than a surprising number of teams expect.

How it fails

Vacuum or compaction lag. WAL or log segments filling a disk. A follower that falls so far behind it cannot catch up. Multi-leader conflicts resolved silently by last write wins. Metadata that grows until start-up takes an hour.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Storage engines and replication topologies fits together

Part	What it does	What goes wrong
Two storage engine families	B-tree engines (many relational databases) keep sorted pages on disk and update them in place.	Treating background cleanup as optional.
Background work is part of the	Both families clean up behind the scenes: vacuum and autovacuum, compaction, checkpointing, log truncation, metadata cleanup.	One synchronous replica that blocks all writes when it dies.
Replication topologies	Leader and followers: one node takes writes and ships a log to followers.	Multi-leader replication with no conflict plan.
Synchronous and asynchronous	Synchronous replication waits for a follower before acknowledging: no loss on failover, but latency, and a dead follower can block writes.	Asynchronous acknowledges at once: fast, but the last writes can be lost.
Metadata and hidden growth	Systems also keep metadata: segment lists, tiered storage indexes, file catalogues.	No alert on replica lag or metadata growth.
Choosing	Ask about the access pattern (point lookups, scans, writes per second), the consistency each feature needs, the growth, and the operations your team can actually run.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Two storage engine families to Background work is part of the / Background work is part of the to Replication topologies / Replication topologies to Synchronous and asynchronous / Synchronous and asynchronous to Metadata and hidden growth / Metadata and hidden growth to Choosing

Incident: A database that refused writes to protect itself

> **Incident:** *A database stopped accepting writes because it was nearing a hard internal limit after cleanup had not kept up.*

```
ERROR:  database is not accepting commands to avoid wraparound data loss in database "app"
HINT:  Stop the postmaster and vacuum that database in single-user mode.
```

This is a simulated rendering of the pattern, not a quotation. The real incidents are Sentry on 20 July 2015 and Mandrill on 4 February 2019; the case files give sources and timelines. The design response is to monitor the distance to the limit, keep cleanup tuned and tested, and keep the ability to take a table out of service quickly.

MARGA by EknathaLabs marga.eknathalabs.com

Interview question

"Why might a write-heavy workload prefer an LSM engine, and what do you watch?" Sequential writes and fast ingestion; watch compaction backlog, read amplification, space amplification and disk I/O.

Common mistakes

- Treating background cleanup as optional
- Multi-leader replication with no conflict plan
- One synchronous replica that blocks all writes when it dies
- No alert on replica lag or metadata growth

Check yourself

What is compaction in an LSM engine?

Answer: Merging sorted files in the background to limit read cost

What is the main risk of multi-leader replication?

Answer: Write conflicts on the same data

In a leaderless quorum system, which condition ensures reads overlap writes?

Answer: R plus W greater than N

What did Honeycomb find in 2024 with one Kafka broker?

Answer: Tiered storage metadata had grown for years without cleanup

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Backup, restore and object storage design

14 minutes. Design backups you have actually restored, protect them from the failure that took the original, and use object storage for what it is good at.

A backup is a hypothesis that you can recover. It becomes a fact only when you restore it, and the failure that takes your data often takes the backups with it. This lesson covers how to design for recovery: what to copy, where, how long it takes, and how to prove it.

What you will learn

- Set backup frequency and retention from RPO and RTO
- Apply the 3-2-1 idea and keep copies independent of the failure
- Test restores and measure real recovery time
- Use object storage well: durability, versioning, lifecycle and consistency

Start from RPO and RTO

RPO says how much data you can lose, so how often you must capture changes. RTO says how long you can be down, so how fast a restore must be. A 5 minute RPO needs continuous log shipping or point-in-time recovery, not a nightly dump. A 1 hour RTO for 2 TB needs a restore path that moves 2 TB in well under an hour: at 200 MB per second, 2 TB takes about 2.8 hours, so a nightly full restore alone fails the target and you need replicas, snapshots or a hot standby.

Kinds of copies

- **Replicas:** protect against node failure, but replicate deletes and corruption instantly.
- **Snapshots:** fast and cheap, often in the same account and failure domain.
- **Backups with point-in-time recovery:** a base copy plus a log of changes, so you can restore to a moment before a mistake.
- **Archives:** long retention for compliance, slow to restore.

Replication is not backup. A script that deletes the wrong thing deletes it everywhere within seconds. In April 2022 a script given the wrong kind of identifier deleted 883 customer sites at Atlassian, and full restoration took until 18 April; the case file explains why restoring many individual sites was a long, careful process. Plan restores for the unit you could lose, not only for a whole database.

Independence

Keep at least one copy that a single event cannot reach: another account, another region and, for important data, another provider. Protect it with separate credentials and write-once or versioned storage so an attacker or a bad script cannot erase it. In May 2024 UniSuper's cloud subscription was deleted by a provisioning misconfiguration that spanned both of its geographies; backups held with another provider were part of how it recovered over days. Whatever name you give it, the principle is the same: a backup in the same account and control plane as the data shares its fate.

Test restores

Schedule regular restore tests, automatically if possible, to a clean environment. Measure the real time, check data integrity (row counts, checksums, an application smoke test), and record it against the RTO. Practise the unpleasant cases: restoring to a point in time, restoring a single table, restoring when the primary credentials are unavailable. A backup job that reports success proves only that a file was written. In August 2023 Toyota's backup system failed along with the main system after a disk space error, so there was no automatic switchover; the case file has the details.

MARGA by EknathaLabs marga.eknathalabs.com

Object storage by design

Object stores hold large amounts of data cheaply and durably, with high availability and simple APIs. Know their shape: objects are immutable blobs addressed by key, not files you edit in place; listing is slower than lookup; per-prefix request rates can matter; consistency models vary by provider, so read the documentation instead of assuming. Use features that match your need:

- **Versioning** to recover from overwrites and deletes
- **Lifecycle rules** to move data to cheaper classes and expire it
- **Object lock or write-once** for protection against deletion
- **Cross-region replication** for disaster recovery, remembering it copies mistakes too
- **Access policies and encryption** with least privilege, and no public access by default

Design keys to spread load and to be listable by prefix (for example tenant and date). Keep a catalogue of metadata elsewhere if you need rich queries.

Cost and retention

Retention is a cost and a risk: long retention of personal data is a liability. Choose per dataset how long to keep it and write it down. Count restore costs too, since retrieving from deep archive tiers can be slow and charged per request or per gigabyte.

How it fails

Backups in the same failure domain. Backups that were never restored. Credentials needed for restore kept only in the system being restored. Retention that deleted the good copy before the corruption was noticed. A restore that takes 3x the RTO.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup, restore and object storage design fits together

Part	What it does	What goes wrong
Start from RPO and RTO	RPO says how much data you can lose, so how often you must capture changes.	Treating replicas as backups.
Kinds of copies	Replicas: protect against node failure, but replicate deletes and corruption instantly.	Backups in the same account and control plane.
Independence	Keep at least one copy that a single event cannot reach: another account, another region and, for important data, another provider.	No protection against deletion of the backups.
Test restores	Schedule regular restore tests, automatically if possible, to a clean environment.	Never timing a restore.
Object storage by design	Object stores hold large amounts of data cheaply and durably, with high availability and simple APIs.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Cost and retention	Retention is a cost and a risk: long retention of personal data is a liability.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Start from RPO and RTO to Kinds of copies / Kinds of copies to Independence / Independence to Test restores / Test restores to Object storage by design / Object storage by design to Cost and retention

Incident: The restore nobody had timed

> **Incident:** *After a destructive change, the team restored from backup and found it would take far longer than the business could accept.*

```
$ pg_restore -j 8 --dbname=app /backups/app-2026-10-28.dump
```

```
... 2h 41m elapsed ...
$ echo "RTO target: 60 minutes"
RTO target: 60 minutes
```

This is a simulated illustration. The real cases with related lessons are the Atlassian site deletion in 2022 and the UniSuper subscription deletion in 2024. The change that followed in a design like this is a warm standby or snapshots for fast recovery, point-in-time recovery for precision, and a scheduled timed restore.

Interview question

"How do you design backups for a 5 TB database with RPO 5 minutes and RTO 1 hour?" Continuous log shipping to an independent account, regular snapshots, a standby for fast failover, a monthly timed restore test, and monitoring on backup age and restore duration.

Common mistakes

- Treating replicas as backups
- Backups in the same account and control plane
- Never timing a restore
- No protection against deletion of the backups

Check yourself

Why is replication not a backup?

Answer: A bad delete or corruption is copied to replicas within seconds

What proves a backup works?

Answer: A restore test with measured time and integrity checks

What does a 5 minute RPO require?

Answer: Continuous change capture such as log shipping or point-in-time recovery

What makes a backup independent?

Answer: Another account and control plane, with separate credentials and deletion protection

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Failure domains, cells and bulkheads

15 minutes. Divide a system so one fault can only reach a bounded part of it, using zones, cells and bulkheads.

A big shared system fails big. The way to bound the damage is to decide in advance what can fail together and keep the rest apart. A failure domain is the set of things one cause can take down. Your design is mostly a list of failure domains and the walls between them.

What you will learn

- Name failure domains: process, host, rack, zone, region, provider, change
- Build cells and bulkheads so a fault cannot spread
- Spot shared dependencies that silently join domains
- Size a cell and route users to one

Failure domains

A process crash, a host, a rack, a power feed, a data centre or availability zone, a region, a cloud provider. There are also non-physical domains: a deploy that goes to everything at once, a shared configuration file, a shared certificate, a shared control plane. The last group causes more large outages than lightning does. Spreading across zones helps against a data centre fault and does nothing against a bad configuration pushed to all zones at once.

Cells

A **cell** is a complete, independent copy of the service for a slice of users. Each cell has its own compute, queues and data. A router sends each customer to exactly one cell. If a cell fails, only its share of users is affected: with 10 cells, a cell fault reaches about 10 percent. Cells also give a safe place to roll out: update one cell, watch, then the next. Capacity grows by adding cells rather than making one cell larger, so the biggest thing you operate stays a known, tested size.

Cell design rules: cells do not call each other; the router is thin and highly reliable (it is the shared part, so keep it simple); a customer's data lives in one cell; keep a hard maximum size and test at it.

Bulkheads

Named after ship compartments. Inside one service, separate resources so one noisy part cannot drain the rest: a separate thread or connection pool per dependency, separate queues for critical and bulk work, separate rate limits per tenant. Without bulkheads, one slow dependency uses all threads and the whole service stalls, including paths that never needed it.

Hidden shared dependencies

Cells and zones are undone by what they share: one database, one DNS provider, one identity service, one configuration store, one deploy pipeline. Draw the dependency graph and mark each node shared or per-cell. Every shared node is a place where cells fail together. In 2021 Roblox's Consul cluster was shared by many services and the outage lasted 73 hours; in 2025 Cloudflare Workers KV relied on a third-party storage provider, and many products failed with it. See the case files.

Static stability

A cell should keep running when its control plane is gone. Pre-provision capacity, cache configuration locally and avoid needing an API call to keep serving. Recovery should not depend on the thing that is broken.

MARGA by EknathaLabs
marga.eknathalabs.com

How it fails

Cells fail when the router fails, when a "global" dependency sneaks in, or when a deploy hits every cell at once. Bulkheads fail when pools are sized so large they still exhaust the host.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Failure domains, cells and bulkheads fits together

Part	What it does	What goes wrong
Failure domains	A process crash, a host, a rack, a power feed, a data centre or availability zone, a region, a cloud provider.	One global config or control plane everywhere at once.
Cells	A cell is a complete, independent copy of the service for a slice of users.	Cells that call each other.
Bulkheads	Named after ship compartments. Inside one service, separate resources so one noisy part cannot drain the rest: a separate thread or connection pool per dependency...	Inside one service, separate resources so one noisy part cannot drain the rest: a separate thread or connection pool per dependency...
Hidden shared dependencies	Cells and zones are undone by what they share: one database, one DNS provider, one identity service, one configuration store, one deploy pipeline.	No hard limit on cell size.
Static stability	A cell should keep running when its control plane is gone.	Counting replicas and calling it isolation.

Flow: Failure domains to Cells / Cells to Bulkheads / Bulkheads to Hidden shared dependencies / Hidden shared dependencies to Static stability

Incident: Three zones, one bad config

> **Incident:** *A service ran in three zones for resilience. A configuration change was applied to all three at once and all three crashed.*

```
$ for z in a b c; do kubectl --context zone-$z get pods -l app=api | grep -c CrashLoop; done
24
24
24
```

This is a simulated illustration. Real cases with the same shape include Cloudflare's November 2025 bot management feature file, which reached the whole proxy fleet, and Microsoft Azure Front Door in October 2025. Spreading across zones did not help because the change was the shared domain. A staged rollout cell by cell would have limited it.

Interview question

"Design for 99.99 percent availability." Start with cells and independent deployment, then ask what is shared and how changes are staged. Zones alone do not reach it.

Common mistakes

- Counting replicas and calling it isolation
- Cells that call each other
- One global config or control plane everywhere at once
- No hard limit on cell size

Check yourself

What is a cell?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: A complete independent copy of the service serving a slice of users

A fault in one of 10 equal cells affects roughly what share of users?

Answer: About 10 percent

What is a bulkhead inside a service?

Answer: Separate resource pools so one slow dependency cannot use everything

Why do zones not protect against a bad config rollout?

Answer: The change is a failure domain that spans all zones

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Blast radius and shuffle sharding

13 minutes. Measure how much a fault can reach, and use shuffle sharding to make one bad client or request unable to hurt the rest.

Blast radius is the share of users or data a single fault can reach. Good designs set a target for it and then show how each fault stays inside it. A poison request, a noisy tenant or a bad deploy should hurt a few, not everyone.

What you will learn

- Define blast radius and set a target
- Reduce it with staged rollout, cells and sharding
- Explain shuffle sharding and compute its effect
- Handle the poison request and the noisy tenant

Blast radius as a number

For each kind of fault write the share affected: a crashed host, a bad deploy, a poisoned input, a lost zone, a bad config. If a bad deploy reaches 100 percent, no amount of zone redundancy gives you a four-nines design. Targets guide choices: a rollout in four stages of 1, 5, 25 and 100 percent limits an undetected bad build to 1 percent for the first detection window.

Sharding to shrink it

If customers are divided among N shards, one customer's fault reaches 1/N of customers. But sharding by customer id alone has a flaw: the unlucky customers sharing the shard with a bad one are always the same.

Shuffle sharding

Instead of mapping a customer to one shard, assign each customer a small random subset of workers, for example 2 out of 8. A poison customer takes down only the workers in its subset. Another customer is hurt only if their whole subset overlaps, and even then they still have a surviving worker if the subsets differ.

Example: 8 workers, each customer gets 2. The number of distinct pairs is 28 (8 choose 2). A given other customer shares both workers with the bad customer with probability 1/28, about 3.6 percent, and loses only one worker with some probability but still has one left in most cases. With plain 4 shards of 2 workers, a bad customer would take out one quarter of customers entirely. Larger fleets give much better numbers: with 100 workers and subsets of 5, there are about 75 million combinations. Clients that retry across their subset will route around the damage.

Shuffle sharding needs clients, or a router, that retry on another worker in their subset, and it assumes faults follow the customer (a poison request, a heavy tenant), not the infrastructure.

Poison requests

A request that crashes the worker that handles it, and which clients then retry onto other workers, spreads like a fire. Defences: shuffle sharding, crash isolation, request validation at the edge, limits on retries, and a "quarantine" that stops a repeatedly failing request from being retried at all. A new unprotected code path with blank policy fields crashed Google's Service Control worldwide in June 2025; see the case file.

Noisy tenants

One tenant using too much hurts others unless there are per-tenant limits, quotas and fair queueing. Quotas that are too coarse fail others; quotas wrong in the other direction fail the tenant. Google's 2020 authentication outage came from an automated quota system that set the quota of a core service to zero because an older system reported its usage as zero. Automated limits need sanity checks and slow, staged application.

MARGA by EknathaLabs marga.eknathalabs.com

Rollout as blast radius control

Stage by cell, region or percentage; watch a bake time with real health signals; stop automatically on regression. The first stage should be small enough to survive being wrong and large enough to carry real traffic.

How it fails

Shuffle sharding fails when faults are in shared infrastructure (a bad deploy to all workers) or when clients do not retry across the subset. Staged rollouts fail when the bake time is too short to catch slow failures, or when health checks measure the wrong thing.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Blast radius and shuffle sharding fits together

Part	What it does	What goes wrong
Blast radius as a number	For each kind of fault write the share affected: a crashed host, a bad deploy, a poisoned input, a lost zone, a bad config.	Per-tenant limits missing or applied without sanity checks.
Sharding to shrink it	If customers are divided among N shards, one customer's fault reaches 1/N of customers.	Percentage rollouts with no bake time.
Shuffle sharding	Instead of mapping a customer to one shard, assign each customer a small random subset of workers, for example 2 out of 8.	Assuming sharding isolates faults that live in shared code.
Poison requests	A request that crashes the worker that handles it, and which clients then retry onto other workers, spreads like a fire.	Global data replication of configuration within seconds with no staging.
Noisy tenants	One tenant using too much hurts others unless there are per-tenant limits, quotas and fair queueing.	Noisy tenants One tenant using too much hurts others unless there are per-tenant limits, quotas and fair queueing.
Rollout as blast radius control	Stage by cell, region or percentage; watch a bake time with real health signals; stop automatically on regression.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Blast radius as a number to Sharding to shrink it / Sharding to shrink it to Shuffle sharding / Shuffle sharding to Poison requests / Poison requests to Noisy tenants / Noisy tenants to Rollout as blast radius control

Incident: One request crashed the fleet

> **Incident:** *A change with blank fields reached binaries that had a new, unprotected feature. A null pointer caused a crash loop everywhere.*

```
$ gcloud logging read 'severity>=ERROR' --limit 3 --format='value(textPayload)'
panic: nil pointer dereference in policy check
panic: nil pointer dereference in policy check
panic: nil pointer dereference in policy check
```

The console lines are a simulated illustration. In the real 12 June 2025 incident the replicated policy data reached all regions within seconds, and the fleet crash-looped; a feature flag and staged rollout of the binary feature would have bounded it. See the case file for the sourced timeline and Google's remediation list.

Interview question

"How do you stop one tenant taking down a shared service?" Per-tenant quotas and fair queueing, shuffle sharding for isolation, and automatic limits on retries from that tenant.

Common mistakes

MARGA by EknathaLabs marga.eknathalabs.com

- Percentage rollouts with no bake time
- Global data replication of configuration within seconds with no staging
- Per-tenant limits missing or applied without sanity checks
- Assuming sharding isolates faults that live in shared code

Check yourself

What does shuffle sharding give each customer?

Answer: A small random subset of workers

With 8 workers and subsets of 2, how many distinct subsets exist?

Answer: 28

Which fault does shuffle sharding NOT help with?

Answer: A bad deploy to every worker

What should a rollout stage include besides a small percentage?

Answer: A bake time with health checks that can stop it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Graceful degradation and circuit breakers

14 minutes. Keep the core working when a dependency is slow or down, by failing fast, serving less and recovering gently.

Every dependency eventually fails. A resilient service decides in advance what it will still do without it. Graceful degradation means serving a reduced but useful result. Circuit breakers stop a service from wasting its own capacity on a dependency that is not answering.

What you will learn

- Rank features as critical, important or optional
- Design fallbacks: cached, default, partial or queued
- Configure a circuit breaker and understand its states
- Avoid fallbacks that fail worse than the failure

Rank what matters

For a shop: checkout and login are critical; search and the product page are important; recommendations, reviews and the loyalty banner are optional. If recommendations are down, show the page without them. This only works if the page is built so a missing part does not break the whole page, and if you decide it before the incident.

Fallback options

- **Serve stale:** return a cached copy, marked as possibly old.
- **Default value:** a generic list instead of a personal one.
- **Partial result:** the answer from the shards that responded, with a flag.
- **Queue for later:** accept the write and process it when the dependency is back.
- **Feature off:** hide the feature and say so.

A fallback is code that runs rarely, so it is likely to be wrong. Test it routinely by switching dependencies off in a staging environment and, when safe, in production during a game day.

Circuit breakers

A breaker wraps calls to a dependency and has three states. **Closed:** calls pass; failures are counted. **Open:** after too many failures or slow calls, calls fail immediately without touching the dependency, for a cool-down time. **Half-open:** after the cool-down, a few trial calls go through; success closes the breaker, failure reopens it.

This protects both sides: the caller stops tying up threads waiting on timeouts, and the dependency gets quiet time to recover. Combine with a timeout (so slowness counts as failure), a bulkhead pool for that dependency, and a fallback for the open state.

Tune on rates, not single failures: for example open when more than 50 percent of the last 20 calls failed or took longer than 2 seconds. Too sensitive and it opens on noise; too sluggish and it never helps.

Fail open or closed

When a safety check cannot be reached, do you allow or deny? A spam filter may fail open (allow mail). A payment fraud check or an authorisation check usually fails closed. Decide per check, write it down, and test it. Cloudflare's 2025 outage came from a component that crashed on an oversized feature file instead of falling back to the previous good file; a safe fallback would have been to keep the last valid configuration.

MARGA by EknathaLabs
marga.eknathalabs.com

Shedding optional work

Under load, drop optional work first: analytics events, background refresh, expensive personalisation. Give each request a priority so overload hits the least important first.

Recovery

Do not fail back all at once. When a dependency returns, ramp traffic, keep the breaker half-open for a while, and avoid synchronized retries.

How it fails

Fallbacks that call the same failing dependency. Breakers that open for everyone because the failures were from one tenant. A degraded mode nobody has tested. Health checks that mark everything unhealthy at once and remove all capacity.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Graceful degradation and circuit breakers fits together

Part	What it does	What goes wrong
Rank what matters	For a shop: checkout and login are critical; search and the product page are important; recommendations, reviews and the loyalty banner are optional.	Putting optional dependencies in the readiness check.
Fallback options	Serve stale: return a cached copy, marked as possibly old.	A fallback that calls the failing service.
Circuit breakers	A breaker wraps calls to a dependency and has three states.	Breakers with no half-open recovery.
Fail open or closed	When a safety check cannot be reached, do you allow or deny?	Fail open or closed When a safety check cannot be reached, do you allow or deny?
Shedding optional work	Under load, drop optional work first: analytics events, background refresh, expensive personalisation.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Recovery	Do not fail back all at once. When a dependency returns, ramp traffic, keep the breaker half-open for a while, and avoid synchronized retries.	Recovery Do not fail back all at once.

Flow: Rank what matters to Fallback options / Fallback options to Circuit breakers / Circuit breakers to Fail open or closed / Fail open or closed to Shedding optional work / Shedding optional work to Recovery

Incident: A health check that removed everything

> **Incident:** *A dependency failed. The health check included it, so every instance reported unhealthy and the load balancer sent traffic to none.*

```
$ curl -s -o /dev/null -w '%{http_code}\n' http://api-1:8080/healthz
503
$ curl -s http://api-1:8080/healthz
{"db":"ok","recommendations":"timeout","status":"unhealthy"}
```

This is a simulated illustration of a common pattern. A health check should answer whether this instance can serve its core job, not whether every optional dependency is fine. After the change, readiness checked only the database and the page degraded when recommendations were down.

Interview question

"Your service depends on five others. One is down. What do you do?" Check which feature depends on it, apply the planned fallback, confirm breakers are open to protect threads, and keep health checks about the instance itself.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Putting optional dependencies in the readiness check
- A fallback that calls the failing service
- Breakers with no half-open recovery
- Never rehearsing the degraded mode

Check yourself

What happens in the open state of a circuit breaker?

Answer: Calls fail immediately without reaching the dependency

Which fallback suits a recommendations widget?

Answer: A default or cached list, or hiding it

What belongs in a readiness check?

Answer: What this instance needs to do its core job

What does half-open do?

Answer: Lets a few trial calls through to test recovery

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Rate limiting, load shedding and admission control

15 minutes. Protect a service from overload with limits at the edge, fair sharing between callers and shedding that keeps useful work flowing.

Overload is not rare. It comes from launches, retry storms, a single heavy customer, a slow dependency and recovery itself. A service that accepts everything delivers nothing: queues grow, latency exceeds timeouts and every request fails. The aim is to serve fewer requests well.

What you will learn

- Choose a rate limit algorithm and where to enforce it
- Share capacity fairly between tenants
- Shed load by priority and use admission control
- Return signals that tell clients to back off

Why overload collapses

When arrival exceeds capacity, queues grow. Wait time passes the client's timeout, so the work is done for a client who has already given up, and the client retries, adding load. Goodput falls toward zero even though servers are busy. This is congestion collapse. Limiting early, before expensive work, keeps goodput high.

Algorithms

- **Fixed window:** count per minute; simple, but allows a burst of double at the window edge.
- **Sliding window:** smoother, more memory.
- **Token bucket:** tokens refill at a steady rate up to a burst size; each request spends one. Allows controlled bursts and is the common default.
- **Leaky bucket:** requests drain at a constant rate, smoothing output.
- **Concurrency limit:** cap in-flight requests rather than rate. Often better for protecting a service, because slow requests hold capacity longer.

Where to enforce

At the edge or gateway (cheap, early), per service (protects itself), and per dependency (protects the callee). A distributed limiter needs shared state, which costs latency and can fail, so decide what happens if the limiter store is down: usually fail open with a local limit as a floor. Docker Hub's pull limits are a public example of limits applied by source address, which surprise teams whose CI jobs share one IP; read the current documentation for the exact numbers.

Fairness

Per-tenant limits stop one customer taking everything. Weighted fair queueing gives each tenant a share and lets idle share be used by others. Pick the key carefully: per IP punishes shared networks, per API key is cleaner.

Load shedding

When over capacity, reject deliberately: drop the lowest priority first (batch, prefetch, analytics), and reject new work rather than abandon work half done. Reject quickly and cheaply; a rejection that costs as much as the work defeats the point. Prefer shedding by queue age: drop requests that have waited longer than the client will wait.

MARGA by EknathaLabs marga.eknathalabs.com

Admission control and clients

Limit concurrency at the entrance based on measured latency, so the service accepts only what it can finish in time. Tell clients: return HTTP 429 (too many requests) or 503 with a Retry-After header, and have clients back off with jitter. A client that ignores the signal needs a hard limit.

Capacity and autoscaling

Autoscaling is slow compared with a spike, and new instances start cold (empty caches, no connections). Do not use it as the only protection. Keep headroom, scale on leading signals, and let limits hold the line while scale-out catches up.

How it fails

A limiter that rejects good traffic because the key is wrong. A limiter store outage that blocks everything. Shedding that rejects critical requests first. A limit set once and never revisited as capacity changed.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Rate limiting, load shedding and admission control fits together

Part	What it does	What goes wrong
Why overload collapses	When arrival exceeds capacity, queues grow.	Counting requests when concurrency is the real constraint.
Algorithms	Fixed window: count per minute; simple, but allows a burst of double at the window edge.	Limits only at the edge, none per dependency.
Where to enforce	At the edge or gateway (cheap, early), per service (protects itself), and per dependency (protects the callee).	A distributed limiter needs shared state, which costs latency and can fail, so decide what happens if the limiter store is down: usually...
Load shedding	When over capacity, reject deliberately: drop the lowest priority first (batch, prefetch, analytics), and reject new work rather than abandon work half done.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Admission control and clients	Limit concurrency at the entrance based on measured latency, so the service accepts only what it can finish in time.	No Retry-After, so clients retry immediately.
Capacity and autoscaling	Autoscaling is slow compared with a spike, and new instances start cold (empty caches, no connections).	Autoscaling as the only defence.

Flow: Why overload collapses to Algorithms / Algorithms to Where to enforce / Where to enforce to Load shedding / Load shedding to Admission control and clients / Admission control and clients to Capacity and autoscaling

Incident: A quota that went to zero

Incident: *An automated quota system reduced a core service's quota to zero because the old system reported its usage as zero.*

```
$ gcloud service-usage quota describe --service=identity --limit=requests
limit: 0
used: 0
status: LIMIT_REACHED
```

The console output is a simulated illustration. In the real 14 December 2020 outage at Google, authentication failed for almost every Google service for about 50 minutes. The lesson is that limits are themselves a dependency that can fail: automated limits need sanity checks, staged change and a floor below which they cannot drop. See the case file.

Interview question

"Design a rate limiter for an API gateway." Ask for the key and the limits, choose a token bucket, keep counters in a fast shared store with local fallback, return 429 with Retry-After, and say what happens when the store is down.

Common mistakes

- Limits only at the edge, none per dependency
- Counting requests when concurrency is the real constraint
- No Retry-After, so clients retry immediately
- Autoscaling as the only defence

Check yourself

What does a token bucket allow?

Answer: Controlled bursts up to the bucket size at a steady refill rate

What is congestion collapse?

Answer: Servers are busy but little useful work completes

Which status and header tell a client to back off?

Answer: 429 or 503 with Retry-After

If the limiter's shared store fails, what is a common choice?

Answer: Fail open with a local limit as a floor

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Multi-region design and global traffic

16 minutes. Choose between active-passive and active-active, set RTO and RPO, and steer traffic with DNS and anycast without creating new single points.

A second region protects against a region failing. It also doubles the things that can go wrong: data in two places, traffic steering, and a failover path that is rarely used. Multi-region is a trade, and it only helps if you can show it works.

What you will learn

- Define RTO and RPO and map them to a pattern
- Compare backup and restore, pilot light, warm standby and active-active
- Handle data across regions
- Steer traffic with DNS, anycast and health checks

RTO and RPO

RTO (recovery time objective) is how long you can be down. **RPO** (recovery point objective) is how much data you can lose, measured in time. A target of RTO 1 hour and RPO 5 minutes rules out nightly backups alone. State both, per service, with the business owner.

Patterns

- **Backup and restore:** hours of RTO, data loss up to the last backup. Cheapest.
- **Pilot light:** core data replicated, compute off until needed. Minutes to hours.
- **Warm standby:** a scaled-down copy running, scaled up on failover. Minutes.
- **Active-active:** both regions serve traffic. Seconds, highest cost and complexity.

Active-passive is simpler, but the standby is untested by real traffic, so failover often fails when first used. Active-active tests both paths all the time, but writes need an answer: one write region per customer (sticky home region), a store with multi-region writes and a conflict rule, or accept stale reads in the second region. Latency across regions (100 to 200 milliseconds between continents) rules out synchronous writes on user paths.

Data is the hard part

Asynchronous replication risks losing the last seconds on failover (RPO above zero). Synchronous across regions costs latency on each write. Pick per dataset. After a failover, decide how to fail back, and how to reconcile writes accepted during the split. Practise it.

Steering traffic

- **DNS (GeoDNS, weighted, health-checked):** simple; limited by TTL and caches that ignore it, so failover takes minutes. Keep records low in TTL if you depend on it.
- **Anycast:** many locations announce the same IP; routing sends users to a nearby one; a failed site withdraws its route. Fast, but it ties you to routing and can send a user's packets to different sites mid-connection.
- **Global load balancers:** health-check regions and shift traffic.

Make sure the steering layer is not the single point of failure. The Facebook outage on 4 October 2021 removed routes to its DNS servers, so the name stopped resolving anywhere. Dyn's DNS attack in 2016 took down many sites that used one DNS provider. Using two independent DNS providers is a standard mitigation. Akamai's edge DNS bug in 2021 shows a vendor change can fail globally in minutes.

MARGA by EknathaLabs marga.eknathalabs.com

Region-wide dependencies

Some services are effectively global in one place. A region can be "up" while the identity or control-plane API it needs runs elsewhere. The October 2025 AWS DynamoDB DNS fault in us-east-1 left many AWS services failing for about 15 hours, which shows how much can hang on one regional endpoint. Map which calls leave your region and test with that region blocked.

Prove it

Run failover drills on a schedule. Verify capacity: the surviving region must take 100 percent of load (or the load you promise), not 50. Track the time from decision to full traffic and compare it with the RTO.

How it fails

A standby that was never exercised. A runbook that depends on the region that failed. DNS caches pinning users to a dead region. Replication lag larger than the RPO. Credentials or configs present only in the primary.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Multi-region design and global traffic fits together

Part	What it does	What goes wrong
RTO and RPO	RTO (recovery time objective) is how long you can be down.	A recovery plan hosted on the failing system.
Patterns	Backup and restore: hours of RTO, data loss up to the last backup.	A failover path used only in drills.
Data is the hard part	Asynchronous replication risks losing the last seconds on failover (RPO above zero).	After a failover, decide how to fail back, and how to reconcile writes accepted during the split.
Steering traffic	DNS (GeoDNS, weighted, health-checked): simple; limited by TTL and caches that ignore it, so failover takes minutes.	One DNS provider.
Region-wide dependencies	Some services are effectively global in one place.	No capacity in the surviving region.
Prove it	Run failover drills on a schedule. Verify capacity: the surviving region must take 100 percent of load (or the load you promise), not 50.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: RTO and RPO to Patterns / Patterns to Data is the hard part / Data is the hard part to Steering traffic / Steering traffic to Region-wide dependencies / Region-wide dependencies to Prove it

Incident: The name that stopped resolving

> **Incident:** *A change on the internal backbone withdrew the routes to the company's DNS servers. The domain stopped resolving and internal tools that used the same network were also unreachable.*

```
$ dig +short example-social.com NS
;; connection timed out; no servers could be reached
$ traceroute -n 203.0.113.53
 1  10.0.0.1  0.4 ms
 2  *
```

The console output is a simulated illustration. In the real 4 October 2021 outage the services were down about six hours; the case file uses Cloudflare's outside view as its source. The design lesson: keep the means to fix a failure, such as out-of-band access and status communication, off the system that is failing.

MARGA by EknathaLabs marga.eknathalabs.com

Interview question

"Design a service for 99.99 percent across regions." Cells in each region, active-active with a home region per customer, health-checked traffic steering with two DNS providers, staged deploys per region, tested failover with survivor capacity, and a clear RPO per dataset.

Common mistakes

- No capacity in the surviving region
- One DNS provider
- A failover path used only in drills
- A recovery plan hosted on the failing system

Check yourself

What does RPO measure?

Answer: How much data, in time, you can afford to lose

Why is active-passive failover risky?

Answer: The standby path is rarely exercised and may fail when first needed

Why use two independent DNS providers?

Answer: One provider failing should not stop name resolution

Two regions at 60 percent load each. One fails. What is the problem?

Answer: The survivor needs 120 percent of its capacity

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. Configuration and control plane change safety

15 minutes. Treat configuration, feature data and control-plane pushes as deployments, with validation, staging and a way back.

Many of the largest recent outages were not hardware failures or attacks. They were a change: a configuration file, a routing update, a policy, a feature list. Changes to data are often pushed faster and wider than code, with fewer checks. Design the change path as carefully as the serving path.

What you will learn

- Explain why configuration is code with a bigger blast radius
- Validate, stage and canary configuration changes
- Keep a last known good copy and a fast rollback
- Separate the control plane from the data plane

Config is code

A configuration value can crash a binary, redirect traffic or disable security as surely as a code change. Yet it is often edited by hand, pushed everywhere at once and reviewed less. Give config the same path as code: version control, review, tests, staged rollout, rollback.

Validate before you push

- **Schema and bounds:** types, ranges, size limits. A file that doubles in size should be rejected before it reaches the proxy.
- **Semantic checks:** does it parse in the real consumer, in a test harness that loads it like production?
- **Diff size:** a change that rewrites half the file is suspicious; require extra review or approval.
- **Compatibility:** old binaries must tolerate new data and new binaries old data; a new field read by a new feature must be safe when blank.

Stage the rollout

Apply to a small cell or percentage, wait a bake time, check real health signals (error rate, latency, crash count), then proceed. Make the rollout automatic and stoppable. Avoid "global replication in seconds": that is an excellent way to deliver a bad value to everyone before a person can react.

Last known good and fail static

When a consumer loads a bad config, it should keep running with the previous valid one and raise an alert, not crash. Fail static: keep serving with the last good data. Pair with a fast rollback that does not depend on the broken system.

Control plane versus data plane

The **data plane** handles user traffic; the **control plane** changes configuration and scale. A good design lets the data plane keep working if the control plane is down or wrong. On 2 November 2023 Cloudflare's control plane and analytics primary site lost power; customers could not change settings, and logs were lost for about two days, yet traffic mostly flowed. The reverse failure is worse: a control plane that pushes a bad value to the data plane everywhere.

Real cases

- Cloudflare, 18 November 2025: a database permissions change made a query return duplicate rows; the bot-management feature file doubled in size, passed a hard limit in the proxy and crashed it on every machine that loaded it.

MARGA by EknathaLabs marga.eknathalabs.com

- Microsoft Azure Front Door, 29 October 2025: incompatible metadata passed health checks, rolled out globally and triggered a latent bug in the data plane; recovery took about 4.5 hours.
- Google Service Control, 12 June 2025: a policy with blank fields met an unprotected new feature.
- Akamai edge DNS, 22 July 2021: a configuration update triggered a component bug; rolled back after about an hour.
- Facebook, 4 October 2021: a backbone configuration change withdrew routes.

Read each case file for the sourced details.

Safe-change checklist

1. Can this change be previewed against production traffic or data?
2. What is the smallest first stage and the stop signal?
3. What is the previous good state and how long does rollback take?
4. What happens if the consumer receives garbage?
5. Is the person fixing it dependent on the thing that is broken?

How it fails

Validation exists but tests a different parser. Staging exists but the pipeline has a "fast path" used in emergencies. Rollback needs the control plane that is down. The change tool and the monitoring share the same dependency.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Configuration and control plane change safety fits together

Part	What it does	What goes wrong
Config is code	A configuration value can crash a binary, redirect traffic or disable security as surely as a code change.	Pushing config to all regions at once.
Validate before you push	Schema and bounds: types, ranges, size limits.	A fast path that skips staging.
Stage the rollout	Apply to a small cell or percentage, wait a bake time, check real health signals (error rate, latency, crash count), then proceed.	Consumers that crash on bad input.
Control plane versus data plane	The data plane handles user traffic; the control plane changes configuration and scale.	On 2 November 2023 Cloudflare's control plane and analytics primary site lost power; customers could not change settings, and logs were...
Real cases	Cloudflare, 18 November 2025: a database permissions change made a query return duplicate rows; the bot-management feature file doubled in size, passed a hard limit in...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Safe-change checklist	Can this change be previewed against production traffic or data?	Rollback depending on the failed system.

Flow: Config is code to Validate before you push / Validate before you push to Stage the rollout / Stage the rollout to Control plane versus data plane / Control plane versus data plane to Real cases / Real cases to Safe-change checklist

Incident: A file that doubled

Incident: *A generated configuration file doubled in size, exceeded a fixed limit in the proxy and the proxy crashed wherever it loaded it.*

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ ls -l /etc/proxy/features.json
-rw-r--r-- 1 root root 2411873 features.json
$ journalctl -u proxy -n 2
proxy: feature count 400 exceeds limit 200
proxy: fatal: aborting
```

This console output is a simulated illustration. The real 18 November 2025 Cloudflare incident is in the case file with its sourced timeline and remediation list. A pre-publish size check, a fail-static loader and a global kill switch are the generic design answers.

Interview question

"How would you make config pushes safe?" Version and review, validate with the real parser, canary by cell, automatic halt on error rate, last known good, and a rollback that works when the control plane is down.

Common mistakes

- Pushing config to all regions at once
- A fast path that skips staging
- Consumers that crash on bad input
- Rollback depending on the failed system

Check yourself

What should a consumer do when it receives an invalid config?

Answer: Keep the last known good config and alert

Why is global replication within seconds risky for config?

Answer: A bad value reaches everyone before anyone can react

What is the data plane?

Answer: The part that handles user traffic

Which check would catch a file that doubled in size?

Answer: A size and shape validation before publish

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. Hidden dependencies and single points of failure

13 minutes. Map what your service really needs, including the tools you use to fix it, and find the single points before an outage does.

A diagram shows the dependencies you remember. Outages come from the ones you forgot: the certificate authority, the DNS provider, the identity service, the status page hosted on the same cloud account, the chat tool you coordinate in. This lesson is a method to find them.

What you will learn

- Build a dependency map including runtime, build, deploy and recovery paths
- Find single points of failure and hard cycles
- Handle third-party and shared platform dependencies
- Keep your recovery tools independent of the system they recover

Four dependency graphs

1. **Serving:** what a user request touches: DNS, CDN, gateway, services, databases, identity, payments.
2. **Build and deploy:** source host, registry, CI, secrets, artifact store, deploy tool. Docker Hub pull limits blocked builds and clusters that pulled from shared addresses.
3. **Operate:** metrics, logs, alerting, paging, on-call chat, runbooks, access (VPN, SSO).
4. **Recover:** backups and their keys, access to restore, DNS to fail over, the status page.

Teams usually map the first and forget the other three.

How to find hidden ones

- Trace one request with distributed tracing and list every hostname called.
- Read egress logs and DNS logs; each external name is a dependency.
- Ask for each: if this is down, what do we lose, and for how long? Rank by impact.
- Block each dependency in staging, one by one, and watch.
- List who holds the credentials needed to recover, and where they are stored.

Cycles

A cycle is when A needs B to start and B needs A. Examples: a cluster that pulls its images from a registry running on that cluster; a secrets store that needs a secret to unlock; a DNS service whose own name must resolve through itself; monitoring that depends on the cluster it monitors. Cycles turn a small fault into a full outage because recovery requires the broken thing. Roblox's 2021 outage was lengthened when monitoring depended on the same Consul cluster that was struggling.

Single points of failure

One of anything: one DNS provider, one certificate expiry date, one engineer with the key, one account, one region, one person who knows the process. Each needs either removal (a second independent path), or acceptance written down with a reason. Expired certificates are a common single point: Microsoft Teams in 2020 and O2 and Ericsson in 2018 are public cases; track expiry dates and automate renewal.

Third parties

You inherit the failures of vendors. Cloudflare Workers KV on 12 June 2025 failed when a third-party storage provider had its own outage, and many Cloudflare products went with it. For each vendor ask: what is the contract SLA, what is our fallback, and can we operate without it for an hour, a day?

MARGA by EknathaLabs marga.eknathalabs.com

Keep recovery independent

Host the status page, the runbooks and the on-call tools outside the platform they describe. Keep break-glass access that works when SSO is down. Keep copies of the critical recovery data in a separate account or provider. Test that you can still reach them with the main system off.

How it fails

A map that is a one-off drawing and is stale in a month. A dependency noted but never tested. A break-glass account nobody has tried this year.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Hidden dependencies and single points of failure fits together

Part	What it does	What goes wrong
Four dependency graphs	Serving: what a user request touches: DNS, CDN, gateway, services, databases, identity, payments.	Status page and runbooks on the failing system.
How to find hidden ones	Trace one request with distributed tracing and list every hostname called.	Vendors with no fallback plan.
Cycles	A cycle is when A needs B to start and B needs A.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Single points of failure	One of anything: one DNS provider, one certificate expiry date, one engineer with the key, one account, one region, one person who knows the process.	Untracked certificate expiry.
Third parties	You inherit the failures of vendors. Cloudflare Workers KV on 12 June 2025 failed when a third-party storage provider had its own outage, and many Cloudflare products...	For each vendor ask: what is the contract SLA, what is our fallback, and can we operate without it for an hour, a day?
Keep recovery independent	Host the status page, the runbooks and the on-call tools outside the platform they describe.	Keep break-glass access that works when SSO is down.

Flow: Four dependency graphs to How to find hidden ones / How to find hidden ones to Cycles / Cycles to Single points of failure / Single points of failure to Third parties / Third parties to Keep recovery independent

Incident: Monitoring that went down with the cluster

Incident: *The monitoring system ran on the platform it watched. When the platform degraded, engineers lost their view of it.*

```
$ curl -s -m 5 https://metrics.internal/api/health
curl: (28) Operation timed out after 5001 milliseconds
$ kubectl get nodes
Unable to connect to the server: net/http: TLS handshake timeout
```

This is a simulated illustration. The Roblox 73-hour case file describes how monitoring that depended on the same cluster made diagnosis harder. The design response is an independent, simple monitoring path for the core platform and an out-of-band way to reach it.

Interview question

"Walk me through hidden dependencies in a service you ran." List one in each of the four graphs, the test you ran to find it and what you changed.

Common mistakes

- Mapping only the serving path
- Status page and runbooks on the failing system

MARGA by EknathaLabs
marga.eknathalabs.com

- Untracked certificate expiry
- Vendors with no fallback plan

Check yourself

Which graph is most often forgotten?

Answer: Operate and recover (monitoring, access, status page)

What is a dependency cycle?

Answer: A needs B to start and B needs A

Where should the status page live?

Answer: On infrastructure independent of the product

How can you discover hidden runtime dependencies?

Answer: Trace requests and read egress and DNS logs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 17. Observability by design

14 minutes. Decide what a system must tell you before it fails: SLIs in the design, golden signals, tracing, and monitoring that survives the outage.

Observability is not a tool you add after launch. It is a property you design in: can an on-call engineer, at 3 a.m., tell what is wrong, how many users are affected and where to look, using only what the system emits? The design question is what to emit, where to keep it, and what happens to your view when the system is down.

What you will learn

- Put SLIs and SLOs into the design, not after it
- Choose signals: golden signals, RED and USE, and which to alert on
- Design traces, logs and metrics so they connect
- Keep monitoring independent and cost under control

Start with the SLI

For each user journey, pick a small number of service level indicators measured as close to the user as practical: the share of requests that succeed and are fast enough, the freshness of data, the share of jobs finished on time. An SLI measured at the load balancer sees more than one measured inside the service, because a crashed service reports nothing. Write the SLO next to the design and make sure the design can measure it.

The signals

- **Golden signals:** latency, traffic, errors, saturation.
- **RED** for services: rate, errors, duration.
- **USE** for resources: utilisation, saturation, errors.

Alert on symptoms users feel (error rate and latency against the SLO, with burn-rate alerts), not on every cause. Use cause metrics (CPU, queue age, connection pool use, replica lag, cache hit rate) for dashboards and diagnosis. Saturation signals that predict trouble are the most valuable: queue age, thread pool use, disk and file descriptor use, distance to a hard limit.

Metrics, logs and traces together

Metrics are cheap and aggregate: use them to detect and to size the problem. Logs are detailed and expensive: use structured fields (request id, tenant, version) so they can be searched. Traces follow one request across services: use them to find where time or errors come from. Connect them by a shared request or trace id in all three, so you can move from a spike on a graph to an example trace to the log lines.

Control cardinality: a label with millions of values (user id, full URL) can overwhelm a metrics system. Put high-cardinality detail in logs and traces, and sample traces sensibly, keeping all errors and slow requests.

Design for the blast radius

Label metrics by cell, region, version and tenant tier so you can see whether a fault is bounded, and compare canary against baseline. A rollout without a per-version error rate cannot be halted automatically. Include change events (deploys, config pushes, flag flips) on the same timeline, because most incidents begin with a change.

MARGA by EknathaLabs
marga.eknathalabs.com

Monitoring that survives

Do not run your only monitoring on the platform it watches. When the platform degrades you lose your view exactly when you need it. Keep a small, simple, independent path for the core signals: external probes from outside your network, a separate metrics store for critical alerts, a separate paging path, and a status page on independent infrastructure. In 2021 Roblox's monitoring depended on the same Consul cluster that was struggling, which made diagnosis harder; in November 2023 a power failure at the primary site for Cloudflare's control plane and analytics meant customers could not change settings and logs were lost for about two days. See the case files.

Alert quality

Every page should be urgent, actionable and linked to a runbook. Review pages weekly and fix or delete noisy alerts. Test alerts: simulate the failure and confirm that the right person is paged in the expected time. An alert that has never fired is untested.

Cost of telemetry

Telemetry can cost as much as the service. Set retention by value: high-resolution metrics for days, downsampled for months, logs hot for days and cold for compliance. Sample traces, drop debug logs by default, and watch the growth in ingestion volume as a signal in itself.

How it fails

Dashboards with fifty graphs and no SLO. Alerts on causes that do not matter. Traces without ids in the logs. Telemetry on the failing platform. A label that explodes cardinality and takes the metrics store down during the incident.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Observability by design fits together

Part	What it does	What goes wrong
Start with the SLI	For each user journey, pick a small number of service level indicators measured as close to the user as practical: the share of requests that succeed and are fast...	Measuring only from inside the service.
The signals	Golden signals: latency, traffic, errors, saturation.	Alerting on causes instead of symptoms.
Metrics, logs and traces together	Metrics are cheap and aggregate: use them to detect and to size the problem.	High-cardinality metric labels.
Monitoring that survives	Do not run your only monitoring on the platform it watches.	Monitoring that depends on the monitored platform.
Alert quality	Every page should be urgent, actionable and linked to a runbook.	An alert that has never fired is untested.
Cost of telemetry	Telemetry can cost as much as the service.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Start with the SLI to The signals / The signals to Metrics, logs and traces together / Metrics, logs and traces together to Monitoring that survives / Monitoring that survives to Alert quality / Alert quality to Cost of telemetry

Incident: Blind during the outage

Incident: *Engineers could not see the state of the platform because the monitoring system ran on it.*

```
$ curl -s -m 5 https://grafana.internal/api/health
curl: (28) Operation timed out after 5001 milliseconds
$ curl -s -m 5 https://probe.example-external.net/check?target=api
{"target":"api","status":"fail","from":"external"}
```

This is a simulated illustration. The external probe still answers when internal monitoring does not, which is the point of an independent path. The Roblox 73-hour case file describes monitoring that shared the struggling cluster; the design response is a small, independent set of probes and alerts for the core signals.

Interview question

"What do you instrument in a new service before launch?" SLIs at the edge with an SLO, golden signals per route, per-version and per-cell labels, trace and request ids across logs, saturation signals for each limit, change events, and an external probe, plus a dashboard and runbook for each alert.

Common mistakes

- Measuring only from inside the service
- Alerting on causes instead of symptoms
- High-cardinality metric labels
- Monitoring that depends on the monitored platform

Check yourself

Which signals are the golden signals?

Answer: Latency, traffic, errors and saturation

Why measure an SLI at the load balancer or edge?

Answer: It sees failures of services that cannot report on themselves

What problem does a high-cardinality metric label cause?

Answer: It can overwhelm the metrics system with too many series

Why keep an independent monitoring path?

Answer: The primary monitoring may fail with the platform it watches

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 18. Security and identity by design

15 minutes. Design for least privilege, short-lived credentials, isolation and recoverable secrets, using how real breaches happened.

Security that is added after the design shows up as a list of exceptions. Security that is designed in shows up as boundaries: who can call what, with which credential, for how long, and what a stolen credential can reach. This lesson takes the design view; it does not replace the security and identity modules, it uses them.

What you will learn

- Draw trust boundaries and design for least privilege
- Prefer short-lived, workload-bound credentials over long-lived keys
- Limit blast radius with isolation and segmentation
- Plan for secret and key compromise, and for detection

Assume a component will be compromised

Design as if any one component, key or account will eventually be misused, and ask what that gives an attacker. If the answer is "everything", the design has a single point of compromise, just as a shared database is a single point of failure. The goals are the same as reliability: bound the blast radius and shorten recovery.

Trust boundaries

Draw the boundaries on your diagram: internet to edge, edge to services, service to service, services to data, admin plane to everything. At each boundary decide who may cross, how they prove who they are, and what is checked. Public access to administration is the commonest mistake: Tesla's cloud account was used for cryptomining in 2017 after attackers reached a Kubernetes administration console that had no password, and found cloud credentials inside a pod. See the case file.

Least privilege

Give each workload only the permissions it needs, on the resources it needs. Wide roles are convenient and expensive. In 2019 an attacker used a server-side request forgery against a misconfigured web application firewall to reach the cloud metadata service, obtained temporary credentials for a role with broad permissions and read data from storage buckets; the case file for Capital One covers this through a published report. Measures that would have narrowed it: restricting the role to what the application needs, enforcing a metadata service version that is harder to abuse, and blocking server-side requests to internal addresses.

Short-lived, bound credentials

A long-lived key is a standing invitation. In 2014 an Amazon access key posted to a public code-sharing site was used to open Uber's S3 data store; the company found out months later. Prefer credentials that expire in minutes or hours, are issued to a workload identity rather than copied into configuration, and are scoped to the task. Scan for secrets in repositories and logs, rotate on a schedule, and make revocation fast and tested.

Segmentation and isolation

Separate environments and accounts (production apart from development), separate data classes, and separate tenants where the risk justifies it. Network policies default to deny. Use separate credentials per cell or per service so one leak does not open everything. Isolation is the security twin of cells and bulkheads.

MARGA by EknathaLabs marga.eknathalabs.com

Protect the control and support planes

Support tools, admin consoles and CI systems often have wide access and weaker controls. Between 28 September and 17 October 2023 an attacker used a service account credential to access Okta's customer support system and download files, including browser session captures with session tokens; 134 customers had files accessed and 5 had sessions hijacked. Treat support and CI as production: least privilege, strong authentication, short sessions, audit logs, and sanitised uploads.

Secrets, keys and recovery

Store secrets in a manager, not in code, images or logs. Use envelope encryption so keys are separate from data and can be rotated. Plan the compromise runbook before you need it: how to find where a secret is used, how to rotate it without downtime, how to revoke sessions, how to confirm no one else is using it.

Detection and audit

Log authentication, authorisation changes, secret access and data export, to a store the application cannot alter. Alert on impossible behaviour: a service account used from a new place, a sudden large read, a new admin. Detection shortens the time attackers have; Uber's key was exposed for months before discovery.

Supply chain

Dependencies and build systems are part of the design. Pin versions, verify sources, keep a software bill of materials, restrict who can publish, and isolate build credentials. Several case files show how one compromised package or build step reached many victims.

How it fails

Long-lived keys in code. One role for everything. Admin interfaces exposed. Support and CI with more access than production. Secrets that cannot be rotated without an outage. Logs that the attacker can edit.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Security and identity by design fits together

Part	What it does	What goes wrong
Assume a component will be	Design as if any one component, key or account will eventually be misused, and ask what that gives an attacker.	One broad role shared by many services.
Least privilege	Give each workload only the permissions it needs, on the resources it needs.	Support and CI tools with wider access than production.
Short-lived, bound credentials	A long-lived key is a standing invitation.	Long-lived keys in code or images.
Protect the control and support	Support tools, admin consoles and CI systems often have wide access and weaker controls.	Exposed admin consoles.
Secrets, keys and recovery	Store secrets in a manager, not in code, images or logs.	Plan the compromise runbook before you need it: how to find where a secret is used, how to rotate it without downtime, how to revoke...
Supply chain	Dependencies and build systems are part of the design.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Assume a component will be to Least privilege / Least privilege to Short-lived, bound credentials / Short-lived, bound credentials to Protect the control and support / Protect the control and support to Secrets, keys and recovery / Secrets, keys and recovery to Supply chain

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A key that opened the data store

Incident: *A long-lived access key was published in a public repository and later used to read a data store.*

```
$ aws sts get-caller-identity --profile found-key
{ "UserId": "AKIA...", "Account": "123456789012", "Arn": "arn:aws:iam::123456789012:user/deploy" }
$ aws s3 ls s3://driver-data --profile found-key
2014-05-12 09:14:07   1388190 drivers.csv
```

This is a simulated illustration with invented identifiers. The real Uber incident in May 2014 is in the case file. A design with short-lived workload credentials, scoped permissions, secret scanning and access logging would have limited both the exposure and the time to detect.

Interview question

"How would you secure service-to-service access in a new platform?" Workload identity with short-lived tokens, mutual authentication, per-service least-privilege policies, default-deny network policy, audit logs, and a rotation and revocation runbook tested in advance.

Common mistakes

- Long-lived keys in code or images
- One broad role shared by many services
- Exposed admin consoles
- Support and CI tools with wider access than production

Check yourself

What does least privilege reduce?

Answer: What a stolen credential can reach

Why prefer short-lived credentials?

Answer: A leaked one expires quickly and is tied to a workload

What was the failure in the Tesla cloud mining case?

Answer: An administration console reachable without a password

Why treat support and CI tools as production?

Answer: They often hold wide access and attackers target them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 19. Capacity planning and cost trade-offs

14 minutes. Plan headroom from measurements, treat cost as a design input, and know which savings quietly spend reliability.

Capacity and cost are two views of one decision: how much to run, and how much margin to carry. Too little and a launch or a failure becomes an outage. Too much and the bill hides the problem. This lesson gives a working method to plan capacity from measured numbers and to make cost a visible trade-off in the design.

What you will learn

- Plan capacity from load tests, peak factors and redundancy rules
- Account for N+1, zone loss and the cold-start problem
- Spot limits that are not CPU: threads, connections, quotas and rate limits
- Make cost trade-offs explicit and avoid hidden reliability cuts

Measure, then plan

Capacity planning starts with measured throughput per unit at the SLO: load test one instance (or one cell) until latency or errors cross your target, and record the requests per second it handles at, say, 60 percent of that point. Then take the forecast peak (average times a peak factor, plus known events such as launches, sales and year-end), and divide. Add redundancy: N+1 so one instance can fail, and zone loss so the remaining zones carry the load. With three zones, losing one means the other two must carry 100 percent, so each can run at most about 66 percent of its capacity before the loss.

Limits that are not CPU

Systems hit other ceilings first: thread counts, file descriptors, database connections, network packets per second, load balancer limits, API rate limits and cloud quotas, connection tracking tables, DNS query rates. Slack's outage on 4 January 2021 followed a surge that overloaded an AWS Transit Gateway; AWS Kinesis on 25 November 2020 failed when a small capacity addition pushed servers over an operating system thread limit; Toyota's plants stopped in August 2023 after a disk space error. List every limit on the request path and watch the distance to each. See the case files.

Headroom and autoscaling

Autoscaling reacts in minutes; a spike arrives in seconds. New instances start cold (empty caches, no warmed connections, long image pulls). Keep headroom for the time it takes to scale, scale on leading signals (queue age, request rate) instead of CPU alone, set sensible minimums, and use limits and load shedding to hold the line meanwhile. Test scale-up under load, not only on paper. When Pokemon GO launched in 2016, traffic reached 50 times the initial target and ten times the worst case estimate; scaling worked because the design could add capacity horizontally and the provider could supply it, which is not something to assume of every dependency.

Forecasting

Use growth trends and business events, and review monthly. Record the assumptions and compare forecast with actual to improve them. Keep a list of lead times: quota increases, hardware, committed spend, procurement. A forecast that is right but needs a six-week lead time is already late.

MARGA by EknathaLabs marga.eknathalabs.com

Cost as a design input

Cost per request or per customer is a number to estimate in the design, as you estimate latency. The big levers are: right-sizing, using committed or discounted capacity for the steady base and on-demand for the peak, storage tiering and lifecycle rules, data transfer (cross-zone and cross-region traffic often costs more than compute), and retention of logs and metrics. Tag resources by team and service so the bill can be read.

Savings that spend reliability

Some savings quietly cut resilience: one zone instead of three, no spare capacity, shorter retention that removes your ability to investigate, dropping a standby, removing the second DNS provider, skipping restore tests. These are legitimate choices only when stated: write the saving and the risk it takes, and have the owner of the SLO agree. A cheaper design that misses the SLO is not cheaper.

Cost spikes are incidents too

A runaway job, a retry storm, a crypto-mining intrusion or an unbounded log can multiply the bill in hours. Set budgets and anomaly alerts, quotas per team, and limits on autoscaling maximums. Tesla's 2017 case, where attackers ran mining software in a cloud account, is a cost incident that began as a security one.

How it fails

A forecast made from average, not peak. A capacity test on a single component while the database is the limit. Autoscaling max set below the real need. A cost cut that removed the margin. A quota hit during a launch.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Capacity planning and cost trade-offs fits together

Part	What it does	What goes wrong
Measure, then plan	Capacity planning starts with measured throughput per unit at the SLO: load test one instance (or one cell) until latency or errors cross your target, and record the...	Planning from averages.
Limits that are not CPU	Systems hit other ceilings first: thread counts, file descriptors, database connections, network packets per second, load balancer limits, API rate limits and cloud...	Running every zone near its limit.
Headroom and autoscaling	Autoscaling reacts in minutes; a spike arrives in seconds.	Watching CPU only.
Cost as a design input	Cost per request or per customer is a number to estimate in the design, as you estimate latency.	Cost cuts with no recorded reliability trade.
Savings that spend reliability	Some savings quietly cut resilience: one zone instead of three, no spare capacity, shorter retention that removes your ability to investigate, dropping a standby...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Cost spikes are incidents too	A runaway job, a retry storm, a crypto-mining intrusion or an unbounded log can multiply the bill in hours.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Measure, then plan to Limits that are not CPU / Limits that are not CPU to Headroom and autoscaling / Headroom and autoscaling to Cost as a design input / Cost as a design input to Savings that spend reliability / Savings that spend reliability to Cost spikes are incidents too

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The limit nobody listed

> **Incident:** *A small capacity addition pushed every server over an operating system limit and the service failed for hours.*

```
$ cat /proc/sys/kernel/threads-max
62000
$ ps -eLf | wc -l
62113
```

The console output is a simulated illustration. In the real AWS Kinesis outage on 25 November 2020 each front-end server kept a thread for every other server, so adding servers pushed each one past the operating system's thread limit. The design response is a list of every limit on the path with an alert on the distance to each, and capacity changes that are tested and staged.

Interview question

"How do you size a service for a launch?" Forecast peak with a margin, load test per unit at the SLO, add N+1 and zone-loss headroom, list non-CPU limits, plan scale-up time and cold start, and agree what is shed when demand exceeds the plan.

Common mistakes

- Planning from averages
- Watching CPU only
- Running every zone near its limit
- Cost cuts with no recorded reliability trade

Check yourself

With three zones, why run each below about 66 percent?

Answer: So two zones can carry the load if one fails

Which of these is a non-CPU limit?

Answer: Thread count, connection tracking or an API quota

What should be written down for a cost saving that reduces redundancy?

Answer: The saving, the risk it takes and who accepted it

Why scale on leading signals such as queue age?

Answer: CPU rises after the problem has begun and new capacity starts cold

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 20. Worked design: a global API with a 99.99 percent SLO

18 minutes. Design a public API for 99.99 percent availability end to end, using cells, regions, safe rollout and degradation, then review it against real outages.

Four nines allows about 4.3 minutes of full unavailability in a 30 day month (0.01 percent of 43,200 minutes). One slow human response uses the whole budget. So the design must prevent most failures from reaching users and must recover automatically for the rest. This lesson walks the six-step method on a concrete problem.

What you will learn

- Turn an SLO into design requirements
- Compose cells, regions, traffic steering, limits and rollout into one design
- Write down the failure modes and the budget each consumes
- Review the design against real outages

1. Clarify

A public REST API for payments-style requests, global customers, 99.99 percent monthly availability measured at the edge as the share of valid requests answered successfully within 1 second. Writes must never be duplicated. Reads may be up to 5 seconds stale. Out of scope: the dashboard and the batch exports.

2. Estimate

Assume 20,000 requests per second at peak, 70 percent reads, 3x peak to average. Average payload 2 KB. Data growth 500 GB per month. One cell sized for 5,000 requests per second at 60 percent utilisation gives 3,000 comfortable. Four cells give 12,000 which is not enough for peak with one lost, so plan 8 cells of 3,000 at steady state: peak 20,000 uses 7 cells, losing one still leaves headroom. Assumptions written down for review.

3. Sketch

- Two or three regions, each with several cells; each customer has a home cell in a home region.
- A thin global router (anycast plus health-checked DNS from two providers) sends a request to a region; a thin cell router sends it to the customer's cell.
- A cell: gateway with rate limits, stateless API services, a database primary with a synchronous replica in a second zone, an asynchronous replica in another region, a queue for non-critical work, a cache.
- Idempotency keys on every write.

4. Deep-dive on the risks

Data: home-region writes with synchronous replication across zones, asynchronous across regions. RPO inside a region is zero; across regions is seconds. Failover for a cell promotes the zone replica automatically with fencing; promotion to another region needs a human decision because it can lose seconds of writes and splits traffic.

Rollout: the biggest outage source. Code and configuration go cell by cell: one canary cell, 15 minute bake with automatic halt on error rate or latency, then cells in groups, one region at a time. No global fast path. Configuration changes use the same pipeline.

Overload: token bucket per customer at the gateway, concurrency limits per service, priority classes (payments critical, exports shed first), load shedding with 429 and Retry-After. Retries only from the edge client with a budget.

MARGA by EknathaLabs marga.eknathalabs.com

Degradation: reads fall back to the replica or cache when the primary is slow; the fraud scorer fails closed for large amounts and fails open with a flag for small ones; webhooks queue.

5. Failure modes and the budget

Failure	Blast radius	Expected effect
Instance crash	one pod	none, retried
Zone loss	one third of a cell	brief errors, automatic promotion
Cell database failure	one cell, about 12 percent of customers	seconds to minutes for those customers
Bad code release	canary cell first	caught in bake, under 1 percent for 15 minutes
Bad config push	canary cell first	same, staged
Region loss	up to half of users	decision-based failover, minutes
Shared dependency (DNS, identity)	all	two DNS providers, cached tokens, static stability

Weighted by frequency, the sum must stay under 4.3 minutes. The honest answer is that a 12 percent cell outage of 20 minutes costs 2.4 minutes of effective global unavailability, so frequency matters; a small cell count and well-tested recovery keep it in budget.

6. Trade-offs

Cells cost more (duplicate infrastructure, harder cross-customer queries). Home regions add latency for far customers. A synchronous replica adds write latency of a millisecond or so within a region. No global transactions across customers. State what the business gave up for the SLO.

Review against real outages

- Cloudflare bot-management file (2025) and Azure Front Door (2025): one change to everything. Your staged rollout and fail-static loaders answer these.
- Google Service Control (2025): replicated policy data crashing every region. Your configuration goes cell by cell.
- AWS DynamoDB DNS (2025): an empty regional DNS record from automation. Your static stability and dependence map apply.
- GoCardless (2017): failover that did not run. Your drills and fencing apply.

How it fails

The design passes review and decays: the router grows features, someone adds a global cache, a fast path appears. Re-run the dependency map and a failover drill every quarter, and track budget burn.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Worked design: a global API with a 99.99 percent... fits together

Part	What it does	What goes wrong
Clarify	A public REST API for payments-style requests, global customers, 99.99 percent monthly availability measured at the edge as the share of valid requests answered...	Failover plans never drilled.
Estimate	Assume 20,000 requests per second at peak, 70 percent reads, 3x peak to average.	Two zones and a load balancer, called four nines.
Sketch	Two or three regions, each with several cells; each customer has a home cell in a home region.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Failure modes and the budget	Weighted by frequency, the sum must stay under 4.3 minutes.	No staged rollout.
Trade-offs	Cells cost more (duplicate infrastructure, harder cross-customer queries).	No written trade-offs.
Review against real outages	Cloudflare bot-management file (2025) and Azure Front Door (2025): one change to everything.	Your staged rollout and fail-static loaders answer these.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Failure modes and the budget / Failure modes and the budget to Trade-offs / Trade-offs to Review against real outages

Incident: A canary that was too big

> **Incident:** *The first stage of a rollout was 25 percent of all traffic. A fault there used most of the quarterly error budget in one afternoon.*

```
$ slo report --window 30d --service api
availability 99.962%  target 99.990%  budget_remaining -180%
```

This console output is a simulated illustration of budget arithmetic, not a named company. The fix was a first stage of one cell (under 5 percent), a bake time with automatic halt, and a policy that a depleted budget freezes risky changes. See the SRE module for error budget policy.

Try the budget arithmetic yourself in the System design lab: estimate the load, then fail each part of a design and watch the budget.

Interview question

"Design an API with 99.99 percent availability." Use the structure above and state the budget math out loud.

Common mistakes

- Two zones and a load balancer, called four nines
- No staged rollout
- Failover plans never drilled
- No written trade-offs

Check yourself

About how many minutes of downtime does 99.99 percent allow in a 30 day month?

Answer: About 4.3

What bounds the damage of a bad release best?

Answer: Staged rollout cell by cell with automatic halt

Why does cross-region failover need a human decision in this design?

Answer: It can lose seconds of writes and split traffic

What should you write at the end of a design?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The trade-offs and what was given up

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 21. Worked design: a status page that survives the outage, and an outage design review

16 minutes. Design the small system that must work when everything else is down, then learn to review a design after an outage.

The status page is the simplest system with the hardest requirement: it must be up exactly when the rest is down. Designing it shows every principle in the module in a small space. After that, we use the same lens to review a real outage design, before and after.

What you will learn

- Derive independence requirements from a failure scenario
- Design a communication path that does not share fate with the product
- Run a before and after design review of an outage using a case file
- Write follow-ups that change the design, not only the runbook

Part 1. The status page

Clarify. Customers and staff must see current state and updates during any outage of our platform, DNS, identity or cloud account. Readers: tens of thousands during an incident. Writers: a few on-call staff. Updates can take a minute to appear. It must work if our SSO is down.

Estimate. Reads at 50,000 requests per second would be a surge; a static page of 50 KB behind a CDN serves that at trivial cost. Writes: a handful an hour.

Sketch.

- A static site generated from a small data file, published to object storage on a different provider and account, behind a CDN on a different DNS zone from the product.
- Updates through a simple authenticated tool with its own accounts (not company SSO) with multi-factor, plus a break-glass path.
- A second publishing path (an API and a documented manual upload) so one tool outage does not stop updates.
- Email and chat notifications from a separate provider.
- Automated probes from outside the platform update the page when checks fail, but a human can override.

Failure modes.

Scenario	Design answer
Our cloud account unavailable	page is on another provider
Our DNS down	status domain on a different registrar and DNS provider
SSO down	separate accounts for publishers, tested quarterly
Publisher tool down	manual upload path documented and drilled
Page itself attacked	static content on a CDN, rate limits
Automated probes wrong	human override, probes labelled as automatic

Trade-offs. A second provider costs a little and adds a process to maintain. Static pages cannot show personalised state. Accept both.

The test. Every quarter, shut off the product's DNS and SSO in a staging copy and publish an update. Time it.

MARGA by EknathaLabs
marga.eknathalabs.com

Part 2. Reviewing an outage design

A design review after an outage asks the same questions as a design review before one. Use this frame on a case file:

1. **Before design:** draw what existed and the dependencies, including build, operate and recover graphs.
2. **Trigger and latent cause:** what changed, and what weakness did it meet?
3. **Why it spread:** which failure domain, shared dependency or missing limit?
4. **Why it lasted:** detection time, diagnosis tools, recovery dependencies, tested rollback?
5. **After design:** the changes, each mapped to a principle (cell, stage, limit, fail static, independence).
6. **What is still open:** residual risk, honestly.

Example: Facebook, 4 October 2021

The case file uses Cloudflare's outside view. Before: DNS servers announced through the company's own backbone; a configuration change on that backbone. Trigger: the change. Latent cause: the DNS servers withdrew themselves from routing when they could not reach the data centres, so the name stopped resolving globally. Why it lasted: tools that depended on the same network. After-design lessons: stage backbone changes with a pre-check that proves access remains, keep out-of-band access, and make DNS survive the loss of its backend. Open: any such choice remains a trade between withdrawing a broken server and keeping a stale one.

Example: S3 in us-east-1, 28 February 2017

Trigger: a debugging command removed more capacity than intended. Latent: two subsystems needed a full restart that nobody had practised at that scale. Why it lasted: restart took longer than expected. After: the tool limits how fast capacity can be removed and a minimum capacity floor; restart paths were practised. A further twist: the status dashboard itself depended on S3, which is exactly the design flaw in Part 1.

How it fails

Reviews that stop at the trigger. Follow-ups that are only "be more careful". A list of 40 actions with no owner and no ordering. Rank by blast radius reduced per effort and track completion.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Worked design: a status page that survives the... fits together

Part	What it does	What goes wrong
Part 1. The status page	Clarify. Customers and staff must see current state and updates during any outage of our platform, DNS, identity or cloud account.	Status page on the same stack.
Part 2. Reviewing an outage design	A design review after an outage asks the same questions as a design review before one.	Review ends at human error.
Incident: The dashboard that needed	This is a simulated illustration. In the real S3 incident of 28 February 2017, AWS reported that the status dashboard's own images were stored in S3, so it could not...	Actions that add procedure but no system change.

Flow: Part 1. The status page to Part 2. Reviewing an outage design / Part 2. Reviewing an outage design to Incident: The dashboard that needed

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The dashboard that needed the broken service

> **Incident:** *During a major storage outage, the page that should have reported it was itself slow to update because it depended on the failing service.*

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://status.example.com
503
```

This is a simulated illustration. In the real S3 incident of 28 February 2017, AWS reported that the status dashboard's own images were stored in S3, so it could not show colour changes and the team used text updates instead. See the case file. The design answer is the independent page in Part 1.

Practise this with a clock in the mock design interview.

Interview question

"You are asked to review a postmortem. What do you look for?" Whether actions reduce blast radius, whether detection and recovery dependencies are addressed, whether owners and dates exist, and whether the fixes get tested.

Common mistakes

- Status page on the same stack
- Review ends at human error
- Actions that add procedure but no system change
- Never testing the communication path

Check yourself

Why must the status page avoid sharing DNS and accounts with the product?

Answer: It must work exactly when the product's infrastructure is down

What is wrong with ending a review at "operator error"?

Answer: It leaves the missing guardrail and slow recovery unchanged

Which review question covers detection and diagnosis?

Answer: Why did it last, and what did our tools depend on?

How should follow-up actions be ranked?

Answer: By blast radius reduced per effort, with owners and dates

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 22. Worked design: a URL shortener

16 minutes. The classic read-heavy design, taken from estimates to key generation, caching, hot links, abuse and failure modes.

A URL shortener looks trivial and is a good test of method: it is read-heavy, needs unique short keys without collisions, has a few extremely hot links, and attracts abuse. The goal here is not the shortest answer; it is a design you can defend with numbers and failure modes.

What you will learn

- Run the six steps on a read-heavy service
- Compare key generation strategies and their failure modes
- Design the cache, the store and the redirect path for hot links
- Add abuse controls and an availability story

1. Clarify

Create a short link for a long URL; redirect on request. Optional custom aliases, expiry and click counts. Redirects must be fast (under 100 ms at the edge) and highly available, because every shared link depends on them. Creation can be slower and can fail briefly. Links must not collide, and a redirect must never go to the wrong target. SLO: 99.95 percent of redirects succeed within 300 ms.

2. Estimate

Assume 100 million new links a month and 100 reads per write. Writes: 100 million divided by about 2.6 million seconds in a month is about 40 per second. Reads: 4,000 per second on average, about 12,000 at a 3x peak. Storage: 100 million links a month at roughly 500 bytes each is 50 GB a month, 600 GB a year, a few TB over five years with replication. The write path is small; the read path and its availability are the design.

A 7 character key in base 62 gives 62 to the power 7, about 3.5 trillion keys, far more than the roughly 6 billion links in five years. Six characters (56 billion) would also do.

3. Sketch

Clients, a CDN or edge cache, redirect service (stateless), a cache, a key-value store partitioned by short key, and a separate create service with a key generator. Redirects return HTTP 301 or 302: 301 is cached by browsers (fewer requests, harder to change or count), 302 is not cached (more requests, easier to control). Pick on purpose: 302 if you count clicks or allow edits.

4. Deep-dive

Key generation. Options: hash the URL and truncate (collisions possible, same URL gives same key, needs collision handling); a counter encoded in base 62 (no collisions, predictable keys, needs a safe distributed counter); pre-generated random keys handed out in blocks from a key service (no collisions, no coordination on the hot path). A common safe choice is each create server leasing a block of ids from a central allocator, then encoding with a keyed permutation so keys are not guessable. If the allocator is down, servers continue until their block runs out; make blocks large enough to outlast an outage.

Store. Lookup by short key only, so a partitioned key-value store with hashed keys fits, with three replicas. Reads dominate and are single-key. Writes are idempotent with a client token so a retry does not create a second link.

Cache. Most traffic goes to a small set of links. A cache in front of the store, plus the CDN for very hot links, takes most reads. Use jittered TTLs, request coalescing and negative caching for missing keys, so a scanner guessing keys does not hit the store. Plan for a viral link: a single key may carry a large share of traffic, so replicate hot keys across cache nodes or serve them at the edge.

MARGA by EknathaLabs
marga.eknathalabs.com

Counts. Counting clicks on the redirect path would slow it. Emit an event to a queue and aggregate asynchronously, accepting that counts lag by seconds.

Abuse. Short links hide destinations, which attackers use. Scan targets against block lists at creation and periodically afterwards, rate limit creation per account and address, offer a preview page for suspicious links, and provide takedown that works through every cache layer.

5. Failure modes

Failure	Effect	Design answer
Cache node lost	misses rise	replicas, coalescing, store sized for cold cache
Store partition down	some keys fail	replicas in other zones, retries with budget
Allocator down	creation stops after block	large blocks, fallback allocator, alert on remaining ids
Viral link	one key hot	edge cache, key replication, per-key shedding
Regional outage	redirects fail	multi-region read replicas, DNS failover, static fallback for hottest links
Bad deploy	wrong redirects	staged rollout, per-version error and redirect checks
Abuse wave	cost and reputation	creation limits, scanning, takedown path

6. Trade-offs

Random keys avoid guessing but lose sortability. 302 costs more traffic but keeps control. Asynchronous counts are cheap but slightly stale. Multi-region replicas cost money and need a plan for writes (one home region for creates, reads everywhere). Say what you chose and what you would revisit at 10x scale.

How it fails

Collisions that send users to the wrong target. A counter that two servers share without coordination. A hot key that overloads one node. A takedown that does not purge the CDN. A cache that was sized for the warm case.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Worked design: a URL shortener fits together

Part	What it does	What goes wrong
Clarify	Create a short link for a long URL; redirect on request.	Creation can be slower and can fail briefly.
Estimate	Assume 100 million new links a month and 100 reads per write.	Spending the interview on key encoding and none on availability.
Sketch	Clients, a CDN or edge cache, redirect service (stateless), a cache, a key-value store partitioned by short key, and a separate create service with a key generator.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Deep-dive	Key generation. Options: hash the URL and truncate (collisions possible, same URL gives same key, needs collision handling); a counter encoded in base 62 (no collisions...	A single counter that is a single point of failure.
Failure modes	.	No story for a viral key.
Trade-offs	Random keys avoid guessing but lose sortability.	Trade-offs Random keys avoid guessing but lose sortability.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: Traffic fifty times the plan

> **Incident:** *A launch brought far more traffic than anyone had planned for, and the backend scaled only because it could add machines quickly.*

```
$ kubectl get hpa redirect
NAME       REFERENCE             TARGETS    MINPODS   MAXPODS   REPLICAS
redirect   Deployment/redirect   95%/60%    10        10        10
```

The console output is a simulated illustration of autoscaling pinned at its maximum. The real related case is the Pokemon GO launch in 2016, where traffic reached 50 times the initial target and ten times the worst case estimate; see the case file. For the shortener, the lesson is to set maximums from the worst case, keep the CDN absorbing hot links, and test scale-up.

Interview question

"Design a URL shortener for 100 million links a month." Use the numbers above, choose block-leased random keys, a partitioned key-value store, a cache and CDN, asynchronous counts, abuse controls and multi-region reads, then walk the failure table.

Common mistakes

- Spending the interview on key encoding and none on availability
- A single counter that is a single point of failure
- 301 without understanding browser caching
- No story for a viral key

Check yourself

With 100 reads per write and 100 million links a month, which path dominates?

Answer: Reads, about 4,000 per second on average

Why use 302 instead of 301 when counting clicks?

Answer: Browsers do not cache 302, so each click reaches you

How can create servers avoid a single point of failure for ids?

Answer: Lease large blocks of ids so they keep working during an allocator outage

MARGA by EknathaLabs marga.eknathalabs.com

What helps a single hot key?

Answer: Edge cache or replicating the key, plus per-key limits

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 23. Worked design: a rate limiter service

16 minutes. Design a shared limiter for an API gateway: algorithm, state, consistency, failure behaviour and how not to cause an outage with it.

A rate limiter protects services and enforces fairness. It sits on every request, so it must be fast and must never become the reason requests fail. The design question is less "which algorithm" than "what do we do when the limiter itself is wrong, slow or down".

What you will learn

- Choose limits, keys and algorithm for a gateway limiter
- Design shared counters and accept approximate accuracy
- Decide fail-open or fail-closed and protect the limiter from itself
- Communicate limits to clients and watch the system

1. Clarify

Limit requests per API key per second and per minute, with different tiers, plus a global protection limit per service. It runs in the gateway on every call. Limit decisions must add under 2 ms. Slightly inaccurate limits are acceptable (a customer allowed 105 instead of 100 for a moment). A limiter outage must not block traffic. Limits are changed by operators and must roll out safely.

2. Estimate

Gateway traffic 50,000 requests per second at peak across 20 gateway nodes, 200,000 active keys. A counter per key and window is small: 200,000 keys by a few counters by about 64 bytes is a few tens of MB, easily in memory. The cost is the network call: asking a shared store for every request is 50,000 operations per second of added latency and a new dependency on the hot path.

3. Sketch

Local limiter in each gateway node (token bucket in memory), plus a shared store (replicated in-memory store) holding counts per key for accurate global limits, synchronised in batches. A configuration service distributes rules. A metrics path records allow, deny and error decisions.

4. Deep-dive

Algorithm. A token bucket per key: tokens refill at the sustained rate up to the burst size; each request spends one. It allows controlled bursts and is cheap. For protecting a service, add a concurrency limit that caps in-flight requests, because slow requests hold capacity.

Where state lives. Three choices: purely local (each node enforces limit divided by nodes; cheap, inaccurate when traffic is uneven); a central store consulted per request (accurate, adds latency and a dependency); or hybrid. The hybrid is common: each node holds a local allowance, and refills it in chunks from the shared store (for example 10 percent of the key's limit at a time). Accuracy is within a chunk, store calls drop by an order of magnitude, and if the store is unreachable the node keeps enforcing its local allowance.

Atomicity. The shared update must be atomic (increment and compare in one operation) to avoid races between gateway nodes. Use the store's atomic operations or a short script, not read then write.

Failure behaviour. If the store is down, fail open to local limits with a conservative floor. Blocking all traffic because the limiter cannot count turns a minor fault into an outage. For sensitive operations such as login attempts, you may choose fail-closed or stricter local limits; decide per rule and write it down.

Protecting the limiter. Cache denies for a short time, apply a cheap pre-filter for obviously abusive sources before the keyed lookup, and bound the work per request. Treat rules as configuration with validation, staged rollout and a floor: an automated or mistaken rule that sets a limit to zero is an outage.

MARGA by EknathaLabs marga.eknathalabs.com

In December 2020 an automated quota system cut the quota of a core Google authentication service to zero because an old system reported its usage as zero, and authentication failed for almost every Google service for about 50 minutes. See the case file.

Client contract. Return 429 with a Retry-After header and rate limit headers showing the limit and remaining count. Document backoff with jitter. Clients that ignore it are limited harder. In December 2021 an automated scaling activity made clients on AWS's internal network flood the devices linking it to the main network, and clients did not back off; congestion lasted hours. A clear contract and client-side backoff matter even inside one company.

5. Failure modes

Failure	Effect	Design answer
Shared store down	accuracy lost	local allowance continues, alert
Store slow	added latency	tight timeout, circuit breaker, skip to local
Bad rule pushed	mass rejections	validation, floor, staged rollout, kill switch
Clock skew	windows misaligned	token bucket with monotonic time
Hot key	one node overloaded	local caching of denies, per-node limits
Gateway restart	local state lost	brief extra allowance acceptable
Retry storm	load amplified	429 with Retry-After, retry budgets

6. Trade-offs

Accuracy against latency: chunked allowances are fast and approximate. Fail-open protects availability and risks abuse during the outage. Per-key limits need many counters; a few tiers with shared limits are cheaper. State these choices.

How it fails

A limit of zero. A store that becomes the slowest call on the path. A distributed counter updated with read-then-write. Clients with no backoff. Operators unable to see which rule denied a request.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: a rate limiter service fits together

Part	What it does	What goes wrong
Clarify	Limit requests per API key per second and per minute, with different tiers, plus a global protection limit per service.	A shared store call on every request with no fallback.
Estimate	Gateway traffic 50,000 requests per second at peak across 20 gateway nodes, 200,000 active keys.	Read then write counters.
Sketch	Local limiter in each gateway node (token bucket in memory), plus a shared store (replicated in-memory store) holding counts per key for accurate global limits...	Limits that can drop to zero.
Deep-dive	Algorithm. A token bucket per key: tokens refill at the sustained rate up to the burst size; each request spends one.	No client guidance on backoff.
Failure modes	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	Accuracy against latency: chunked allowances are fast and approximate.	Fail-open protects availability and risks abuse during the outage.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: A quota that read zero

Incident: *An automated process set a limit to zero from a bad reading, and the limiter obeyed.*

```
$ ratectl show rule identity-core
limit_per_minute: 0
source: automated-quota-sync
last_usage_reading: 0
```

This is a simulated illustration of the pattern. The real event is the Google authentication quota outage of 14 December 2020, in the case file. The guards are a floor under every limit, a sanity check against the previous value, staged application and a kill switch.

Interview question

"Design a rate limiter for an API gateway." Ask for keys and limits, choose token bucket plus a concurrency limit, use local allowances refilled in chunks from a shared atomic store, fail open with a floor, return 429 with Retry-After, validate and stage rule changes, and monitor decisions.

Common mistakes

- A shared store call on every request with no fallback
- Read then write counters
- Limits that can drop to zero
- No client guidance on backoff

Check yourself

What does a local allowance refilled in chunks give you?

Answer: Fewer store calls and continued enforcement if the store is down

Which failure behaviour is common for a limiter when its store is unreachable?

Answer: Fail open with a conservative local limit

Why must counter updates be atomic?

Answer: Concurrent gateway nodes would otherwise race and miscount

MARGA by EknathaLabs marga.eknathalabs.com

What should a rule change process include?

Answer: Validation, a floor, staged rollout and a kill switch

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 24. Worked design: a notification fan-out system

16 minutes. Deliver one event to millions of devices, with fan-out strategies, priority queues, provider limits, deduplication and retry control.

A notification system turns one event ("new message", "payment received", "outage") into many deliveries over email, push and chat. The hard parts are fan-out, uneven load, third-party limits and not sending the same message twice or a million at once to a recovering dependency.

What you will learn

- Design fan-out on write and fan-out on read, and when to mix them
- Use priorities, queues and per-provider limits
- Make delivery idempotent and avoid duplicate sends
- Protect providers and the system from retry storms

1. Clarify

Producers publish events with a type, an audience and a payload. The system delivers to each recipient's chosen channels. Critical notices (security alerts, payment failures) are time sensitive; marketing can wait. Users set preferences and quiet hours. Duplicates are annoying, and for some notices harmful. SLO: 99 percent of critical notifications are handed to a provider within 10 seconds, bulk within 30 minutes.

2. Estimate

50 million users, 5 notifications per user per day: 250 million a day, about 2,900 per second average, bursts to 10x on events (a flash sale or an outage notice). A single broadcast to 20 million users must complete in minutes: 20 million over 600 seconds is about 33,000 per second sustained, far above the average. Provider limits apply: email and push providers cap per-second sends and per-recipient rates, so the egress rate is set by the providers, not by our fleet.

3. Sketch

Event intake API, an event stream, a fan-out service that expands audiences into per-recipient messages, priority queues per channel, channel workers that call providers, a preference and deduplication store, a delivery status store, and a dead-letter queue.

4. Deep-dive

Fan-out strategy. For targeted notices (one user), expand immediately. For huge audiences (all users), expanding to 20 million messages at intake would flood the queue and delay critical traffic. Expand in batches by partition of the user table, as background jobs that are rate limited, and keep them off the critical lane. That is fan-out on write for most cases, with controlled batching for broadcasts. Where a recipient pulls their feed (an in-app inbox), fan-out on read avoids storing millions of copies; hybrids store for normal users and read for celebrity-sized audiences.

Priority lanes. Separate queues and workers for critical, normal and bulk traffic, with their own capacity so a marketing blast cannot delay a security alert. That is a bulkhead. Workers pull from critical first.

Provider limits and backpressure. Each channel worker has a rate limiter matched to the provider's limits and a circuit breaker. If the provider slows or returns errors, the breaker opens, messages wait in the queue, and bulk is delayed before critical. Use a second provider for critical channels when the cost justifies it.

Idempotency and dedup. Give each notification a unique id (event id plus recipient plus channel). Before sending, record the id in the dedup store in the same step as claiming the send; a redelivery from the queue finds the id and skips. Providers that accept an idempotency key should receive it. Accept that some paths are at-least-once and make duplicates harmless where possible.

MARGA by EknathaLabs marga.eknathalabs.com

Retries. Retry transient failures with exponential backoff and jitter, a cap, and a retry budget per provider. Invalid addresses and unsubscribed users are not retried; failures go to a dead-letter queue with an owner. In January 2019 a retry storm from one application overloaded CoreDNS in Zalando's Kubernetes clusters, pods ran out of memory and DNS went down for over an hour: the same pattern applies to workers retrying a slow provider while also overloading name resolution. See the case file.

Preferences and quiet hours. Check preferences at send time, not only at intake, because they change. Defer non-critical messages during quiet hours using a delayed queue, and spread the morning wake-up burst with jitter.

5. Failure modes

Failure	Effect	Design answer
Provider outage	deliveries stall	breaker, queue, secondary provider for critical
Broadcast floods queue	critical delayed	separate lanes, rate-limited expansion
Worker crash mid-send	possible duplicate	dedup record, idempotency key
Poison message	worker loops	attempt cap and dead-letter queue
Queue backlog	stale notices	age alerts, expire old bulk messages
Preference store down	wrong sends	cached preferences, fail safe to critical only
Retry storm	provider and DNS overload	jitter, budget, limited concurrency

6. Trade-offs

At-least-once delivery with dedup is simpler than exactly-once and occasionally duplicates. Separate lanes cost idle capacity. A second provider costs money and integration work. Expiring stale bulk messages protects users from a flood of old notices at the cost of undelivered ones. State which you chose.

How it fails

A broadcast that starves critical alerts. Duplicate sends after a worker restart. A provider limit exceeded so the account is throttled or suspended. An expired credential for a provider. A dead-letter queue nobody reads.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Worked design: a notification fan-out system fits together

Part	What it does	What goes wrong
Clarify	Producers publish events with a type, an audience and a payload.	Ignoring provider rate limits and quotas.
Estimate	50 million users, 5 notifications per user per day: 250 million a day, about 2,900 per second average, bursts to 10x on events (a flash sale or an outage notice).	Expanding a broadcast at intake.
Sketch	Event intake API, an event stream, a fan-out service that expands audiences into per-recipient messages, priority queues per channel, channel workers that call...	One queue for all priorities.
Deep-dive	Fan-out strategy. For targeted notices (one user), expand immediately.	Retrying without a budget.
Failure modes	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	At-least-once delivery with dedup is simpler than exactly-once and occasionally duplicates.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: A retry storm against shared infrastructure

Incident: *One application retried aggressively and overloaded a shared service that many others needed.*

```
$ kubectl top pod -n kube-system -l k8s-app=kube-dns
NAME                       CPU(cores)   MEMORY(bytes)
coredns-6d4b75cb6d-x7k2p   1998m        1.9Gi
$ kubectl get events -n kube-system | grep OOMKilled | head -2
Warning  OOMKilled  pod/coredns-6d4b75cb6d-x7k2p  Container exceeded memory limit
```

The console output is a simulated illustration. The real case is Zalando on 7 January 2019, in the case file. For a notification system, the response is a per-provider retry budget, jittered backoff, concurrency limits per worker pool, and keeping bulk traffic behind critical traffic so a storm cannot take shared dependencies down.

Interview question

"Design a notification system for 50 million users." Cover event intake, fan-out strategy, priority lanes, per-provider limits, dedup ids, retries with budgets, preferences, and the failure table.

Common mistakes

- One queue for all priorities
- Expanding a broadcast at intake
- Retrying without a budget
- Ignoring provider rate limits and quotas

Check yourself

Why separate queues for critical and bulk notifications?

Answer: A bulk blast cannot delay critical messages

How do you avoid sending a duplicate after a worker restart?

Answer: Record a unique notification id and skip if it exists

What sets the real maximum send rate?

Answer: The provider limits and the rate limiters you place in front of them

MARGA by EknathaLabs
marga.eknathalabs.com

Where do messages that keep failing go?

Answer: A dead-letter queue with an owner

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 25. Worked design: a metrics and log pipeline

16 minutes. Ingest, store and query telemetry at scale without letting the pipeline become the outage, with cardinality, backpressure and retention.

Observability data is a high-volume, write-heavy workload with a special property: you need it most when everything else is failing, which is when it is hardest to keep running. The design must handle load spikes, bound cost and keep a path for the core signals alive.

What you will learn

- Estimate ingest rates and storage for metrics and logs
- Design a pipeline with buffering, backpressure and sampling
- Control cardinality and retention
- Keep alerting working when the platform is not

1. Clarify

Collect metrics (numbers over time), logs (events) and traces from 2,000 services across several clusters. Dashboards for engineers, alerts to on-call, long-term retention for audits. Alerts on core service health must fire within 1 minute of a problem. Dashboards may lag by seconds. Losing some debug logs is acceptable; losing the alerts is not. Cost is a major constraint.

2. Estimate

Metrics: 2,000 services times 50 instances times 500 series each is 50 million active series. At a 15 second scrape interval that is about 3.3 million samples per second. At roughly 2 bytes per sample after compression, that is about 6.6 MB per second, 570 GB per day, before replication and indexes. Logs: 100,000 lines per second at 300 bytes is 30 MB per second, 2.6 TB a day. Logs cost far more than metrics, so retention and sampling decide the bill.

3. Sketch

Agents or collectors on each node, a regional buffer (a log or queue), ingest workers that write to a time-series store for metrics, an indexed store for recent logs and cheap object storage for old logs, a query layer, an alert evaluator, and a notification path. Separate, small, independent path for core alerts.

4. Deep-dive

Buffering and backpressure. Put a durable buffer between collectors and storage so a slow store does not drop data or push back into applications. Bound the buffer; when it fills, drop the least valuable data first (debug logs, high-volume low-priority metrics), never application traffic. Collectors must never block the service they observe.

Cardinality control. The number of distinct series is the main cost and failure driver. One label with millions of values (user id, request id, full URL) can multiply series and take the store down. Enforce limits per team and per metric, reject or aggregate offending labels at ingest, alert on series growth, and keep high-cardinality detail in logs and traces. Allocate quotas per tenant so one team cannot starve others (a noisy-neighbour bulkhead).

Sampling and tiering. Keep all error traces and slow traces, sample the rest. Downsample metrics (raw for 15 days, 5 minute for 13 months). Keep logs hot for a few days, warm for weeks, and cold in object storage with lifecycle rules. Sampling decisions should be consistent so a trace is complete.

Partitioning. Shard series by a hash of the series key across ingest nodes with replicas for durability; shard logs by time and tenant. Plan compaction and index maintenance (see the storage engine lesson): an ingest surge can build compaction debt that slows queries.

MARGA by EknathaLabs marga.eknathalabs.com

Alerting path. The alert evaluator should read from a fast, replicated store and be redundant across zones. Provide a minimal independent path: external probes against the user-facing endpoints, a separate small metrics store for a handful of core SLIs, a separate paging provider. Dead-man's-switch alerts (an alert that fires if the pipeline stops sending its heartbeat) catch silent failure of the monitoring itself. In November 2023 a power failure at the primary site for Cloudflare's control plane and analytics meant logs were lost for about two days and customers could not change settings; see the case file for the sourced account.

Multi-region. Collect locally and ship upward, so a region keeps its own view if the central site is unreachable. Replicate the core alerting data to a second region.

5. Failure modes

Failure	Effect	Design answer
Ingest surge	buffers fill	bounded buffer, drop low priority, auto scale
Cardinality explosion	store overload	per-tenant limits, reject at ingest, alert on growth
Storage node loss	gaps or slow queries	replicas, quorum writes, repair
Query of death	store overloaded	query limits, timeouts, per-user quotas
Pipeline silently stops	blind on-call	dead-man's switch, external probes
Region loss	no central view	regional stores, replicated core alerts
Cost overrun	budget cut leads to data loss	quotas, retention tiers, ingestion alerts

6. Trade-offs

Dropping debug logs under pressure saves the pipeline and loses detail. Long raw retention is costly; downsampling loses spikes. Central storage is easy to query and a single point; regional storage is resilient and harder to query globally. Independent core alerting is another system to run. State the choices.

How it fails

The monitoring platform on the same cluster as the workload. Alerts that depend on the store that is overloaded. No alert for missing data. Logs ingested without limits. Dashboards that run expensive queries on every refresh.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Worked design: a metrics and log pipeline fits together

Part	What it does	What goes wrong
Clarify	Collect metrics (numbers over time), logs (events) and traces from 2,000 services across several clusters.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Estimate	Metrics: 2,000 services times 50 instances times 500 series each is 50 million active series.	Monitoring that shares the failing platform.
Sketch	Agents or collectors on each node, a regional buffer (a log or queue), ingest workers that write to a time-series store for metrics, an indexed store for recent logs and...	No alert for missing data.
Deep-dive	Buffering and backpressure. Put a durable buffer between collectors and storage so a slow store does not drop data or push back into applications.	No cardinality limits.
Failure modes	.	Unbounded buffers.
Trade-offs	Dropping debug logs under pressure saves the pipeline and loses detail.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: Logs lost with the site

Incident: *The primary data centre for a control plane and analytics system lost power, and logs were lost for days.*

```
$ logctl status --region primary
primary:   unreachable
secondary: standby (not receiving)
last_replicated_event: 2023-11-02T08:41:12Z
```

The console output is a simulated illustration. In the real incident on 2 November 2023, a power failure at a Flexential data centre in Portland took down the primary site of Cloudflare's control plane and analytics; customers could not change settings and logs were lost for about two days. See the case file. Design responses are standby capacity that actually receives data, regular failover tests, and keeping the customer data path independent of the control plane.

Interview question

"Design a metrics system for 2,000 services." Estimate series and samples, buffer and shed, control cardinality, tier retention, keep an independent core alert path with a dead-man's switch, and discuss multi-region.

Common mistakes

- No cardinality limits
- Monitoring that shares the failing platform
- Unbounded buffers
- No alert for missing data

Check yourself

What is the main driver of metric store cost and failure?

Answer: The number of distinct series (cardinality)

What should a collector do if storage is slow?

Answer: Never block the service; buffer and drop low-value data if needed

MARGA by EknathaLabs marga.eknathalabs.com

What does a dead-man's switch alert detect?

Answer: The monitoring pipeline going silent

Why keep high-cardinality detail in logs and traces?

Answer: Metrics series multiply with each distinct label value

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 26. Worked design: a multi-tenant Kubernetes platform

17 minutes. Share clusters between teams safely: isolation levels, quotas, upgrades, networking, and keeping the platform's own dependencies from becoming the outage.

An internal platform offers teams a place to run workloads without each building their own infrastructure. The design questions are isolation (how much can one tenant harm another), operability (how the platform itself is changed safely), and blast radius (how many tenants one mistake reaches).

What you will learn

- Choose a tenancy model: namespaces, clusters per team, or cells
- Apply quotas, policies and network rules for isolation
- Plan cluster upgrades and add-on changes as staged rollouts
- Identify the platform's shared dependencies

1. Clarify

150 teams, 3,000 services, mixed criticality (customer-facing and batch). Teams self-serve namespaces, deployments and ingress. The platform team owns clusters, networking, ingress, DNS, observability agents and policies. Requirements: one team cannot starve or read another; platform upgrades cause no tenant downtime; cluster loss affects a bounded share; SLO for the control plane API 99.9 percent, for tenant workloads set by tenants.

2. Estimate

3,000 services at an average of 6 pods is 18,000 pods. A single cluster is practical up to a few thousand nodes and tens of thousands of pods, but limits (API server load, etcd size, DNS, network policies, rate limits) tighten earlier. Design for about 8 clusters of 400 to 600 nodes in 2 regions, not one very large cluster: failure of one cluster affects roughly an eighth of workloads and upgrades can be staged.

3. Sketch

Several clusters per region, each with the same platform layer (ingress, DNS, policy engine, observability agents, declarative delivery of platform config). A fleet controller assigns tenants to clusters. A global traffic layer routes by hostname. Shared services (identity, registry, secrets) are outside the clusters.

4. Deep-dive

Tenancy model. Namespace-per-team in shared clusters is cheap and has weaker isolation; cluster-per-team gives strong isolation at high cost and operational load; a middle path groups tenants into a small number of clusters by risk tier (critical, standard, batch), so a noisy batch tenant cannot share a failure domain with payments.

Isolation controls. Resource quotas and limit ranges per namespace; requests and limits on every pod; priority classes so critical workloads preempt batch; pod security standards enforced by admission policy; network policies default deny with explicit allows; separate service accounts with least-privilege RBAC; node pools or taints for sensitive or noisy workloads; per-tenant limits on the number of objects (a tenant that creates a million objects can overload the API server and etcd).

Platform dependencies. Cluster DNS, the CNI network plugin, the ingress controller, the policy webhooks, the registry and the identity provider are shared by every tenant in a cluster. A failing admission webhook can block all deployments; a broken CNI breaks all networking. Reddit's 2023 outage came from a Kubernetes upgrade from 1.23 to 1.24 that removed a node label the Calico route reflectors depended on; cluster networking broke and the site was down for about 314 minutes. Zalando's 2019 DNS overload and Preply's 2020 DNS and conntrack problems show the same theme: platform add-ons are single points per cluster. Give each a dependency review, redundancy, resource limits and a rollback.

MARGA by EknathaLabs
marga.eknathalabs.com

Upgrades. Treat upgrades as staged rollouts: a canary cluster with synthetic and real low-risk tenants, a bake time, then clusters in groups across regions. Read deprecation notes and run checks for removed APIs and labels before each upgrade, and test the add-ons against the new version in a copy of production. Keep the ability to rebuild a cluster from declarative config quickly instead of repairing in place, and keep the previous control plane version rollable where the platform supports it.

Control plane safety. Back up etcd (or the managed equivalent) and test restores; protect the API server from unbounded clients; keep admission webhooks fail-safe with timeouts and exclusions for system namespaces.

Cost and capacity. Use cluster autoscaling with maximums, per-team showback, and headroom for losing a zone (see the capacity lesson). Spread each service across zones with topology constraints and disruption budgets so node drains during upgrades do not take a service down.

5. Failure modes

Failure	Effect	Design answer
Noisy tenant	neighbours starved	quotas, limits, priority, node pools
Admission webhook fails	deployments blocked	timeouts, fail-safe policy, exclusions, redundancy
Bad cluster upgrade	cluster-wide outage	canary cluster, checks, staged groups, rebuild path
DNS or CNI overload	cluster networking fails	sized replicas, limits, node-local caches, load tests
API server overload	control plane unusable	client limits, per-tenant object caps, priority and fairness
Registry rate limit or outage	pods cannot start	internal mirror, cached images
Cluster loss	up to an eighth of workloads	multi-cluster deployment for critical services

6. Trade-offs

More clusters mean smaller blast radius and more operational work (more upgrades, more config). Shared clusters are efficient and demand strong policy. Strict network policy slows onboarding and prevents lateral movement. A mirror of images costs storage and avoids third-party rate limits (Docker Hub limits pulls by source address). State the tiering you chose.

How it fails

One shared cluster for everything. Add-ons without redundancy or limits. Upgrades done everywhere in a weekend. Admission policies that block recovery actions. Tenants with no quotas.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: a multi-tenant Kubernetes platform fits together

Part	What it does	What goes wrong
Clarify	150 teams, 3,000 services, mixed criticality (customer-facing and batch).	One giant shared cluster.
Estimate	3,000 services at an average of 6 pods is 18,000 pods.	No tenant quotas or object limits.
Sketch	Several clusters per region, each with the same platform layer (ingress, DNS, policy engine, observability agents, declarative delivery of platform config).	Check the Common mistakes list below this diagram for what usually goes wrong here.
Deep-dive	Tenancy model. Namespace-per-team in shared clusters is cheap and has weaker isolation; cluster-per-team gives strong isolation at high cost and operational load; a...	Upgrading every cluster at once.
Failure modes	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	More clusters mean smaller blast radius and more operational work (more upgrades, more config).	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: An upgrade that broke networking

> **Incident:** *A control plane upgrade changed a label that the network layer depended on, and cluster networking failed.*

```
$ kubectl get nodes -l kubernetes.io/role=control-plane
No resources found
$ calicoctl node status
BGP peers: 0 established, 12 down
```

The console output is a simulated illustration. The real Reddit incident on 14 March 2023 is described in the case file with a sourced timeline: an upgrade of an older cluster from 1.23 to 1.24, a removed node label, broken Calico route reflection, about 314 minutes of downtime. The design answers are pre-upgrade checks for removed labels and APIs, canary clusters, and a faster rebuild.

Interview question

"Design a multi-tenant Kubernetes platform." Choose tenancy tiers and several clusters per region, apply quotas, policies and network isolation, treat add-ons as single points, stage upgrades, back up and restore, and keep critical services spread across clusters.

Common mistakes

- One giant shared cluster
- Treating add-ons as invisible
- No tenant quotas or object limits
- Upgrading every cluster at once

Check yourself

Why use several clusters rather than one very large one?

Answer: A fault or upgrade affects a bounded share and limits are easier to stay under

What can one failing admission webhook do?

Answer: Block deployments for every tenant in the cluster

How should cluster upgrades be rolled out?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Canary cluster, bake time, then staged groups with pre-checks

What protects the API server from one tenant?

Answer: Per-tenant object limits, client limits and priority and fairness

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 27. Worked design: an authoritative DNS service

16 minutes. Design the name service that everything else depends on: anycast, zone data safety, DDoS resilience and the discipline of not changing everything at once.

DNS is the dependency under every other dependency. If your names stop resolving, nothing else matters, and your tools to fix it may depend on DNS too. Designing an authoritative service means designing for extreme availability, abuse and careful change.

What you will learn

- Estimate query load and design anycast serving
- Keep zone data consistent and changes safe
- Resist floods and avoid a single provider
- Plan for the failure of the service that serves its own names

1. Clarify

Serve authoritative answers for customer zones (or a company's internal and public zones). Queries are small UDP requests, with TCP and DNSSEC as required. Customers update records through an API and expect changes to appear globally within a minute. Availability target: 100 percent of queries answered in practice; a published SLO of 99.999 percent. A wrong answer is worse than no answer for many records.

2. Estimate

100 million zones, 5 billion queries per day on average is about 58,000 per second, with attacks capable of 100x that. Each response is 100 to 500 bytes. A single modern server can answer hundreds of thousands of simple queries per second, so ordinary load is small; the design is driven by attack traffic, geographic latency and availability. Zone data: 100 million zones at about 1 KB each is about 100 GB, which fits in memory per node with compact encoding.

3. Sketch

A control plane (API, validation, change pipeline, zone database) and a data plane (name servers in many locations announcing the same addresses by anycast). Zone data flows one way from control to data plane through a replicated distribution system, and each name server serves from a local copy so it can keep answering if the control plane is gone.

4. Deep-dive

Anycast. Many sites announce the same IP addresses; routing sends each client to a nearby healthy site, and a failing site withdraws its routes. This absorbs floods (attack traffic spreads across sites) and cuts latency. Publish several name server addresses from different network prefixes and different providers so one routing problem or one provider cannot remove them all. In October 2016 an attack on Dyn, partly from devices infected with the Mirai botnet, made many popular sites unreachable for users on the US East Coast; sites that used only that provider had no alternative. See the case file.

Static stability. Each site keeps its last known good zone data and keeps answering when the control plane or distribution is down. Health checks withdraw a site from anycast only when it cannot answer correctly, and never withdraw everything: a rule that withdraws a site if it cannot reach its backend can withdraw every site when the backend fails. Facebook's outage on 4 October 2021 removed routes to its DNS servers after a configuration change on the backbone, and the name stopped resolving; the case file uses Cloudflare's outside view.

Zone data safety. Treat zone changes as deployments. Validate syntax and semantics before publishing, run checks for dangerous changes (deleting an apex record, large deletions, a change that would remove most records), stage by site or percentage, watch for answer anomalies, and keep

MARGA by EknathaLabs
marga.eknathalabs.com

versioned snapshots for instant rollback. In July 2021 a software configuration update triggered a bug in a DNS component of Akamai's Secure Edge CDN, and some customer websites were unavailable for up to about an hour until it was rolled back. See the case file.

Software diversity and safe releases. A bug in name server software hits every site that runs it. Run releases through canary sites, bake time and automatic rollback on answer error rate; consider running two independent implementations at different sites so one bug cannot take all of them.

Abuse and floods. Rate limit per source with response rate limiting to blunt reflection abuse, drop malformed queries early, size for peak attack traffic with headroom, and use scrubbing for volumetric attacks. Keep TTLs on your own critical records sensible: short TTLs speed failover and increase query load; long TTLs hide problems for hours.

Consistency and propagation. Choose the freshness promise: changes visible within a minute at all sites. Distribute through a versioned log, show the version each site serves, and alert on sites that lag. Secondary DNS providers receive transfers or API updates; make sure the second provider is genuinely independent.

Its own dependencies. The control plane, its authentication, and the tools used to repair the service must not depend on the service's own names. Keep break-glass access by IP address.

5. Failure modes

Failure	Effect	Design answer
Volumetric attack	sites saturated	anycast spread, headroom, scrubbing, rate limits
Bad zone change	wrong or empty answers	validation, staged rollout, snapshots, rollback
Control plane down	no updates	sites serve last good data
Bad software release	all sites crash	canary, bake, rollback, second implementation
Site withdrawal cascade	no sites left	never withdraw all, health rules reviewed
Provider outage	some addresses dark	multiple prefixes and providers
Lag between sites	inconsistent answers	version tracking, lag alerts

6. Trade-offs

Short TTLs help failover and increase load. Two software implementations double maintenance and cut correlated failure. Aggressive health-based withdrawal removes sick sites quickly and risks cascades. A second provider costs money and demands careful zone sync. Choose and write it.

How it fails

A single provider or single prefix. A health check that depends on the control plane. Zone changes pushed everywhere at once. The repair tools depending on the names that are down.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: an authoritative DNS service fits together

Part	What it does	What goes wrong
Clarify	Serve authoritative answers for customer zones (or a company's internal and public zones).	Zone changes applied everywhere at once.
Estimate	100 million zones, 5 billion queries per day on average is about 58,000 per second, with attacks capable of 100x that.	Health checks that withdraw every site together.
Sketch	A control plane (API, validation, change pipeline, zone database) and a data plane (name servers in many locations announcing the same addresses by anycast).	All name servers on one provider or prefix.
Deep-dive	Anycast. Many sites announce the same IP addresses; routing sends each client to a nearby healthy site, and a failing site withdraws its routes.	Repair tools that need the broken service.
Failure modes	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	Short TTLs help failover and increase load.	A second provider costs money and demands careful zone sync.

Flow: Clarify to Estimate / Estimate to Sketch / Sketch to Deep-dive / Deep-dive to Failure modes / Failure modes to Trade-offs

Incident: When the name servers disappear

Incident: *The routes to the DNS servers were withdrawn and the name stopped resolving anywhere.*

```
$ dig +short NS example-service.net
;; connection timed out; no servers could be reached
$ bgpctl show ip bgp 198.51.100.0/24
network not in table
```

The console output is a simulated illustration with documentation addresses. The real events in the case files are Facebook on 4 October 2021 (routes to its DNS servers withdrawn after a backbone configuration change), the Dyn attack of 21 October 2016, and the Akamai edge DNS bug of 22 July 2021. The shared design lessons are static stability, staged change, independent providers and recovery tools that do not need the broken names.

Interview question

"Design an authoritative DNS service." Use anycast across diverse prefixes and providers, local zone copies, validated and staged zone changes, rate limiting, canary software releases, static stability, and an independent control plane and break-glass access.

Common mistakes

- All name servers on one provider or prefix
- Health checks that withdraw every site together
- Zone changes applied everywhere at once
- Repair tools that need the broken service

Check yourself

What does anycast give a DNS service?

Answer: Nearby routing, spread of attack traffic and withdrawal of failing sites

What should a name server do if the control plane is unreachable?

Answer: Keep serving its last known good zone data

Why use more than one DNS provider?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: A single provider outage or attack should not remove all name servers

Which zone change deserves extra protection?

Answer: A large deletion or removal of apex records

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 28. Worked design: a chat and messaging service

18 minutes. A walk through a chat design: connections, delivery guarantees, ordering, presence and what happens when a node or region fails.

Chat looks simple until you count connections and promise delivery. Millions of users hold a connection open, messages must arrive once and in order, and people expect it to work on a weak phone network. This worked design follows the six steps from the design method: clarify, estimate, sketch, deep-dive, failure modes and trade-offs.

What you will learn

- Size a connection-heavy service from users and message rates
- Design delivery with acknowledgements, retries and idempotent message ids
- Keep message order within a conversation
- Plan for gateway loss, reconnect storms and regional failure

1. Clarify

Direct messages and small groups up to 200 people, text and attachments by link, delivery receipts, history sync across devices, and a target of 99.95 percent availability. Messages must not be lost and must stay in order within a conversation. Global ordering across conversations is not needed. End-to-end encryption is out of scope for this design but changes where servers can index content.

2. Estimate

Assume 50 million daily users, with 10 million connected at peak. Each user sends 40 messages a day, so 2 billion messages a day, about 23,000 a second on average and 70,000 at a 3x peak. A message is about 1 KB with metadata, so 2 TB a day and around 700 TB a year before replication. Connections matter more than requests: with 50,000 connections per gateway, 10 million connections need about 200 gateways plus headroom.

3. Sketch

Clients keep a WebSocket to a gateway. The gateway authenticates and forwards messages to a chat service. The service writes each message to a partitioned log and a message store, then finds the recipients' gateways through a session registry and pushes to them. Offline users receive a push notification and sync from the store when they reconnect.

4. Deep-dive: ids, order and delivery

Each client generates a message id, so a retry cannot create a duplicate. Within a conversation, the server assigns a sequence number, using the conversation id as the partition key so one node orders it. The client acknowledges each message by sequence number, and the server retries unacknowledged messages. On reconnect the client says "my last sequence is 812" and the server sends what follows.

5. Deep-dive: presence and fan-out

Presence changes often and few people need every change. Update presence from heartbeats with a time-to-live, and send it only to users who have the contact open. For a group of 200, fan out by looking up each member's gateway. For very large channels, switch to pull: clients fetch from the log, and the server pushes only a light "new messages" signal.

MARGA by EknathaLabs marga.eknathalabs.com

6. Failure modes

Failure	Effect	Design answer
Gateway dies	50,000 clients disconnect	clients reconnect with jittered backoff, session registry expires entries by ttl
Reconnect storm	the others overload	admission control, backoff with jitter, slow ramp
Chat node dies	one partition stalls	leader election per partition, replicas
Store partition slow	writes lag	log accepts first, store catches up, alert on lag
Region lost	users there offline	users homed per region, failover with replicated logs
Duplicate send	double message	client ids make writes idempotent

7. Trade-offs

Per-conversation ordering is cheap, global ordering is not worth its cost. Pushing through gateways gives low latency but needs a registry that can be briefly wrong, so delivery relies on acknowledgements, not on the registry. Storing every message forever is simple, but old data should move to cheaper storage. Say what you would revisit at ten times the scale.

Sketching the capacity numbers

```
$ python3 -c "dau=50_000_000; per_day=dau*40; avg=per_day/86_400; print(per_day, round(avg), round \
    (avg*3), 'gateways', 10_000_000/50_000)"
2000000000 23148 69444 gateways 200.0
```

Two lines of arithmetic give the load: about 23,000 messages a second on average and around 70,000 at peak, with 200 gateways for ten million connections.

How it fails

Duplicate messages after a retry. A reconnect storm that overloads the gateways. Order lost because two nodes handle one conversation. A registry that says a user is online when they are not, so messages are pushed into the void.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Worked design: a chat and messaging service fits together

Part	What it does	What goes wrong
Clarify	Direct messages and small groups up to 200 people, text and attachments by link, delivery receipts, history sync across devices, and a target of 99.95 percent...	Trying to guarantee global ordering.
Estimate	Assume 50 million daily users, with 10 million connected at peak.	Ignoring connection counts and sizing only by requests.
Deep-dive: ids, order and delivery	Each client generates a message id, so a retry cannot create a duplicate.	Relying on the session registry for delivery guarantees.
Deep-dive: presence and fan-out	Presence changes often and few people need every change.	No jittered backoff on clients.
Trade-offs	Per-conversation ordering is cheap, global ordering is not worth its cost.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Sketching the capacity numbers	Two lines of arithmetic give the load: about 23,000 messages a second on average and around 70,000 at peak, with 200 gateways for ten million connections.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Deep-dive: ids, order and delivery / Deep-dive: ids, order and delivery to Deep-dive: presence and fan-out / Deep-dive: presence and fan-out to Trade-offs / Trade-offs to Sketching the capacity numbers

Incident: When everyone reconnected at once

> **Incident:** *A gateway cluster restarted after a deployment. Every client reconnected within seconds, which overloaded authentication and the session registry, so many clients retried and made the overload last longer.*

```
$ kubectl get pods -l app=gateway
NAME         READY   STATUS             RESTARTS
gateway-7    0/1     CrashLoopBackOff   6
$ curl -s auth.internal/metrics | grep login_rate
login_rate 41000/s (capacity 9000/s)
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration of a reconnect storm. Clients that back off with jitter, a server that sheds excess logins, and a slow, staged restart keep a deployment from turning into an outage.

Interview question

"Design a chat service for 50 million daily users." Use the numbers above, choose WebSocket gateways, partition by conversation for ordering, client ids for idempotency, acknowledgements for delivery, and walk the failure table, especially reconnect storms.

Common mistakes

- Ignoring connection counts and sizing only by requests
- Relying on the session registry for delivery guarantees
- No jittered backoff on clients
- Trying to guarantee global ordering

Check yourself

Why partition messages by conversation id?

Answer: One node can order the messages of a conversation cheaply

How does the design prevent duplicate messages after a retry?

Answer: The client supplies a message id so the write is idempotent

MARGA by EknathaLabs marga.eknathalabs.com

What helps most against a reconnect storm?

Answer: Jittered backoff on clients and admission control on the server

Why drive delivery with acknowledgements instead of the session registry?

Answer: The registry can be briefly wrong, while acknowledgements confirm real receipt

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 29. Worked design: a news feed and timeline

18 minutes. Design a feed for a social product: fan-out on write versus read, ranking, hot accounts and cache failures.

A feed shows each user a recent, ranked list of posts from people they follow. Reading is far more common than writing, and a few accounts have millions of followers. The whole design is a choice between doing work when a post is written and doing it when the feed is read. This lesson works through that choice and its failure modes.

What you will learn

- Compare fan-out on write with fan-out on read
- Handle accounts with millions of followers
- Combine ranking with a cached candidate list
- Protect the feed from cache and ranking failures

1. Clarify

Users follow others, post short items with media links, and open a home feed of recent posts from those they follow. The feed should load in under 300 milliseconds and show a new post within a few seconds. A user may follow up to 5,000 accounts. Perfect freshness is not required, and a slightly stale feed is better than an empty one.

2. Estimate

Assume 100 million daily users who open the feed 10 times a day, so a billion feed reads a day, about 12,000 a second and 35,000 at peak. Posts are 100 times fewer: 10 million a day, about 120 a second. Each post is delivered to the feeds of followers; with an average of 200 followers that is 2 billion feed insertions a day, about 23,000 a second.

3. Fan-out on write

When a post is created, the system pushes its id into the feed list of every follower, held in a cache such as a sorted set per user. Reads are then very fast: take the latest ids and fetch the posts. The cost is write amplification, which is acceptable for ordinary accounts and a problem for an account with 20 million followers.

4. Fan-out on read

The feed is built when the user opens it: fetch recent posts from each followed account and merge them. Writes are cheap, but reads are expensive and slow for users who follow many accounts. This suits rarely active users, where precomputing is wasted.

5. The hybrid

Use write fan-out for normal accounts and read-time merge for celebrity accounts. At read time, take the user's precomputed list, then merge in recent posts from the few high-follower accounts they follow. A threshold such as 100,000 followers decides which path an account uses. Skip write fan-out for inactive users and build their feed on first return.

6. Ranking and failure modes

Rank a few hundred candidates with a lightweight model, and fall back to reverse-chronological order if the ranker is slow.

Failure	Effect	Answer
Feed cache node lost	some feeds empty	rebuild from the post store on demand, serve stale

MARGA by EknathaLabs marga.eknathalabs.com

Failure	Effect	Answer
Celebrity posts	write spike	read-time merge for those accounts
Ranker slow	feed latency	strict timeout, chronological fallback
Post store slow	partial feeds	show cached items and mark gaps
Delete or block	removed items remain	filter at read time against a deletion set

7. Trade-offs

Write fan-out spends storage and write work to buy fast reads. The hybrid avoids the worst case at the price of two code paths. Ranking improves engagement but needs a fallback, so the feed never goes blank. Say how freshness, cost and complexity trade, and what you would change for ten times the users.

Counting feed insertions

```
$ python3 -c "inserts=10_000_000*200; print(inserts, round(inserts/86_400), 'celebrity post insert \
    s', 20_000_000)"
2000000000 23148 celebrity post inserts 20000000
```

The average account is fine for write fan-out, but one post from an account with 20 million followers would create 20 million inserts at once. That is why celebrities use read-time merge.

How it fails

A celebrity post floods the write path. The cache is lost and every feed rebuilds at once. The ranker adds seconds of latency. Deleted posts keep showing. A user who follows thousands of accounts times out.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Worked design: a news feed and timeline fits together

Part	What it does	What goes wrong
Clarify	Users follow others, post short items with media links, and open a home feed of recent posts from those they follow.	Rebuilding every feed at once after a cache loss.
Estimate	Assume 100 million daily users who open the feed 10 times a day, so a billion feed reads a day, about 12,000 a second and 35,000 at peak.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Fan-out on read	The feed is built when the user opens it: fetch recent posts from each followed account and merge them.	No fallback when ranking is slow.
The hybrid	Use write fan-out for normal accounts and read-time merge for celebrity accounts.	Fan-out on write for every account.
Trade-offs	Write fan-out spends storage and write work to buy fast reads.	Ranking improves engagement but needs a fallback, so the feed never goes blank.
Counting feed insertions	The average account is fine for write fan-out, but one post from an account with 20 million followers would create 20 million inserts at once.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Fan-out on read / Fan-out on read to The hybrid / The hybrid to Trade-offs / Trade-offs to Counting feed insertions

Incident: A post from an account with millions of followers

> **Incident:** *A very popular account posted during a busy hour. The system pushed the post into millions of feed lists in a burst, which queued behind ordinary writes and delayed everyone's feed by several minutes.*

```
$ queuectl depth feed-fanout
```

MARGA by EknathaLabs · marga.eknathalabs.com

```
queue          depth      oldest
feed-fanout    18400000   412s
$ queuectl top-producers feed-fanout
account:88231  17900000 pending
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Moving high-follower accounts to read-time merge, and putting their fan-out in a separate queue with limits, keeps one post from delaying everyone.

Interview question

"Design a news feed." Estimate reads and writes, compare write and read fan-out, propose the hybrid with a follower threshold, add ranking with a chronological fallback, and discuss celebrity posts and cache failure.

Common mistakes

- Fan-out on write for every account
- No fallback when ranking is slow
- Ignoring deletes and blocks
- Rebuilding every feed at once after a cache loss

Check yourself

What is the main cost of fan-out on write?

Answer: Write amplification for accounts with many followers

Why use a hybrid approach?

Answer: Ordinary accounts use write fan-out and very popular accounts use read-time merge

What should happen when the ranker is slow?

Answer: Fall back to a chronological feed after a timeout

How are deleted posts hidden from cached feeds?

Answer: Filter against a deletion set at read time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 30. Worked design: video upload and streaming

18 minutes. From upload to playback: chunked upload, transcoding queues, storage tiers and a CDN, with the failure modes of each stage.

A video service is mostly a pipeline and a cache. Large files go in, workers convert them into several formats, and a content delivery network serves millions of viewers. The design is about moving big data reliably, spending compute only where needed, and keeping playback smooth when a stage is slow.

What you will learn

- Design resumable, chunked uploads
- Build a transcoding pipeline with queues and retries
- Pick storage tiers and use a CDN for delivery
- Handle hot videos, failed jobs and cost control

1. Clarify

Creators upload videos up to several gigabytes. Viewers watch on phones and televisions with different speeds. A video should be playable within minutes of upload. Availability of playback matters more than instant availability of new uploads. Cost matters, because storage and delivery are the biggest bills.

2. Estimate

Assume 500,000 uploads a day averaging 500 MB, which is 250 TB a day of originals. Each video is converted into five renditions that add about 1.5 times the original, so total storage grows by about 625 TB a day. Playback dominates traffic: 100 million views a day at 100 MB each is 10 PB a day, so a CDN carries almost all of it.

3. Resumable upload

The client splits the file into chunks of a few megabytes and uploads each with an id and checksum. If the connection drops, it asks which chunks exist and sends only the missing ones. Once all chunks are present, the service assembles the file in object storage and records the upload as complete.

4. Transcoding pipeline

A completed upload creates a job in a queue. Workers split the video into segments, convert each segment into every rendition in parallel, then stitch the results and produce a playlist. Jobs are idempotent and have timeouts, so a failed worker is simply retried. Poison files that always fail go to a dead-letter queue for review.

5. Storage and delivery

Keep originals in cheap, durable storage and renditions in standard storage. Serve playback through a CDN with segments of a few seconds, so players can switch quality as bandwidth changes. Popular videos stay in the CDN, and rarely watched ones are fetched from the origin on demand.

MARGA by EknathaLabs marga.eknathalabs.com

6. Failure modes

Failure	Effect	Answer
Upload drops	partial file	resume by chunk, checksums
Worker dies	job stalls	visibility timeout and retry
Bad file	job loops	retry limit, dead-letter queue
Queue backlog	new videos delayed	autoscale workers, priority for small jobs
Origin slow	playback stalls	CDN shielding, request coalescing
Viral video	origin load spike	pre-warm the CDN, cache at several tiers

7. Trade-offs

More renditions give better playback and cost more storage and compute. Converting every video on upload is simple but wasteful for videos nobody watches, so some services convert popular formats first and others on demand. A longer CDN time to live saves origin load but delays takedowns, so provide a purge path.

A job record in the pipeline

```
job: transcode
video_id: v_8f31a2
source: s3://originals/v_8f31a2/source.mp4
segments: 120
renditions: [240p, 360p, 480p, 720p, 1080p]
attempts: 1
max_attempts: 3
timeout_seconds: 900
on_final_failure: dead_letter
output: s3://renditions/v_8f31a2/
```

The job states its retry limit, timeout and where it goes if it fails for good. This is how a pipeline stays reliable without humans watching every job.

How it fails

A big upload restarts from zero after a network drop. A corrupt file makes workers retry forever. The queue backs up after a burst of uploads. A viral video overloads the origin. A takedown does not reach the CDN.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: video upload and streaming fits together

Part	What it does	What goes wrong
Clarify	Creators upload videos up to several gigabytes.	Serving video from the origin instead of a CDN.
Estimate	Assume 500,000 uploads a day averaging 500 MB, which is 250 TB a day of originals.	No path to purge a removed video from the cache.
Transcoding pipeline	A completed upload creates a job in a queue.	Unlimited retries for failing jobs.
Storage and delivery	Keep originals in cheap, durable storage and renditions in standard storage.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	More renditions give better playback and cost more storage and compute.	Check the Common mistakes list below this diagram for what usually goes wrong here.
A job record in the pipeline	The job states its retry limit, timeout and where it goes if it fails for good.	A job record in the pipeline The job states its retry limit, timeout and where it goes if it fails for good.

Flow: Clarify to Estimate / Estimate to Transcoding pipeline / Transcoding pipeline to Storage and delivery / Storage and delivery to Trade-offs / Trade-offs to A job record in the pipeline

Incident: A file that never finished converting

> **Incident:** *One uploaded file had a corrupt header. Workers picked it up, crashed, and the job returned to the queue, over and over. Each retry consumed a worker, and the backlog delayed other creators.*

```
$ queuectl stats transcode
depth 5400   in_flight 120   retries_today 38812
$ queuectl top-retries transcode
v_9c20b7  attempts 412
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A retry limit with a dead-letter queue turns a poison file into one reviewed item, instead of a long-running drain on shared workers.

Interview question

"Design video upload and streaming." Estimate storage and egress, use chunked resumable upload, a queue of idempotent transcode jobs with a dead-letter queue, tiered storage and a CDN, and walk the failure table.

Common mistakes

- Uploading a large file in one request
- Unlimited retries for failing jobs
- Serving video from the origin instead of a CDN
- No path to purge a removed video from the cache

Check yourself

Why upload in chunks?

Answer: A dropped connection resumes with the missing chunks only

What should happen to a file that keeps failing conversion?

Answer: After a retry limit it goes to a dead-letter queue for review

Why does a CDN matter most here?

Answer: Playback traffic is far larger than upload traffic

Why make transcode jobs idempotent?

Answer: A retry after a worker failure must not corrupt or duplicate output

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 31. Worked design: a distributed key-value store

19 minutes. Partitioning, replication, quorum reads and writes, and failure handling, put together into one store.

A key-value store is the building block under many systems. Designing one forces you to combine ideas from earlier lessons: consistent hashing, replication, quorums and failure detection. The point is to be explicit about the guarantees you give and what each one costs in latency and availability.

What you will learn

- Combine partitioning and replication in one design
- Choose quorum sizes for reads and writes
- Handle node failure with hinted handoff and repair
- State the consistency you provide and why

1. Clarify

Operations are get, put and delete by key, with values up to a few kilobytes. Latency target is a few milliseconds at the 99th percentile. The store must keep working when a node or an availability zone fails, and it can choose between strict and eventual consistency per request. Range scans are out of scope.

2. Estimate

Assume 10 TB of data, 200,000 operations a second at peak and 80 percent reads. With three replicas the raw storage is 30 TB. At 2 TB per node that is 15 nodes, and 25 to 30 with headroom for growth and rebalancing. Each node handles about 20,000 operations a second including replica traffic, well within a single machine.

3. Partitioning

Hash each key onto a ring and give each node many virtual positions. A key belongs to the first node clockwise and to the next two distinct nodes as replicas, placed in different zones. Adding a node moves only a small slice of keys. Virtual positions even out the load and let a stronger machine take more of the ring.

4. Replication and quorums

With N replicas, a write waits for W acknowledgements and a read asks R replicas. If R + W is greater than N, a read overlaps the latest write. For N = 3, W = 2 and R = 2 gives strong reads of recent writes. W = 1 and R = 1 is fast and may return stale data. Make this a per-request setting.

5. Failure handling

When a replica is down, a coordinator stores hints for it and replays them when it returns. A background anti-entropy process compares replicas using hash trees and repairs differences. Concurrent writes to one key need a rule: last write wins by timestamp is simple but can lose data, while version vectors keep both versions for the application to merge.

MARGA by EknathaLabs marga.eknathalabs.com

6. Failure modes

Failure	Effect	Answer
Node down	fewer replicas	hinted handoff, quorum still met
Zone down	one replica lost per key	replicas spread across zones
Network partition	two sides diverge	quorum blocks the minority for strict requests
Slow node	tail latency	hedged reads, drop slow replica from the quorum
Hot key	one node overloaded	cache, split the key, shed load
Clock skew	wrong last write wins	logical clocks or version vectors

7. Trade-offs

Larger quorums give stronger reads and slower, less available operations. Last write wins is simple and can lose a write. Version vectors preserve conflicts but burden the application. State what you offer, such as read-your-writes within a session, and what you do not, such as global transactions.

A table definition with quorum settings

```
store: sessions
replicas: 3
placement: spread_across_zones
consistency:
  default_write: quorum   # W = 2
  default_read: quorum    # R = 2
  fast_read: one          # R = 1, may be stale
conflict_resolution: version_vector
repair:
  hinted_handoff_window: 3h
  anti_entropy_interval: 6h
```

With three replicas and quorum reads and writes, a read always overlaps the latest successful write. The fast read option trades freshness for speed on request.

How it fails

A quorum cannot be met during a partition. Hints are lost when a node is down longer than the window. Last write wins hides a lost update. A hot key overloads its replicas. Rebalancing after adding nodes saturates the network.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: a distributed key-value store fits together

Part	What it does	What goes wrong
Clarify	Operations are get, put and delete by key, with values up to a few kilobytes.	Placing all replicas in one zone.
Estimate	Assume 10 TB of data, 200,000 operations a second at peak and 80 percent reads.	Choosing quorum sizes without thinking about R + W > N.
Replication and quorums	With N replicas, a write waits for W acknowledgements and a read asks R replicas.	Relying on wall clocks to decide the latest write.
Failure handling	When a replica is down, a coordinator stores hints for it and replays them when it returns.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	Larger quorums give stronger reads and slower, less available operations.	State what you offer, such as read-your-writes within a session, and what you do not, such as global transactions.
A table definition with quorum	With three replicas and quorum reads and writes, a read always overlaps the latest successful write.	Rebalancing at full speed in production.

Flow: Clarify to Estimate / Estimate to Replication and quorums / Replication and quorums to Failure handling / Failure handling to Trade-offs / Trade-offs to A table definition with quorum

Incident: Adding nodes during a busy hour

> **Incident:** *A team added several nodes to the cluster at peak traffic. The data movement used so much network bandwidth that normal reads and writes slowed down, and some requests missed their quorum timeouts.*

```
$ kvctl rebalance status
moving: 41 partitions   rate: 1.9 Gbit/s   (limit: none)
$ kvctl latency --p99
reads 38 ms (normal 4 ms)   writes 55 ms (normal 6 ms)
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Rate-limiting rebalancing, moving a few partitions at a time and scheduling it outside peak hours keep growth from harming live traffic.

Interview question

"Design a distributed key-value store." Use consistent hashing with virtual nodes, N=3 replicas across zones, quorum reads and writes with R + W > N, hinted handoff and anti-entropy, a conflict rule, and discuss failures and trade-offs.

Common mistakes

- Choosing quorum sizes without thinking about R + W > N
- Placing all replicas in one zone
- Rebalancing at full speed in production
- Relying on wall clocks to decide the latest write

Check yourself

When does a read overlap the latest write?

Answer: When R + W is greater than N

Why spread replicas across zones?

Answer: A zone failure should lose at most one replica of each key

What does hinted handoff do?

Answer: Holds writes for a down replica and replays them when it returns

Why avoid relying on clocks for last write wins?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Clock skew can pick the wrong write and lose data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 32. Worked design: a payments ledger

19 minutes. Correct money movement: double-entry records, idempotent requests, transactions and reconciliation, designed to survive retries and failures.

In a payments system, a wrong answer is worse than a slow one. Money must not be created, lost or counted twice, even when requests are retried and servers fail halfway. The design leans on a few simple ideas: record every movement as immutable entries that balance, make every request idempotent, and check the books against the outside world regularly.

What you will learn

- Model money as double-entry, append-only records
- Make payment requests idempotent
- Use transactions and state machines correctly
- Reconcile with external systems and handle failures

1. Clarify

Customers pay merchants. The service creates a payment, authorises it with an external processor, captures it, and later refunds some or all of it. Correctness comes first: no double charges, no lost payments, and a complete audit trail. Latency target is under one second for a payment. Availability must not cost correctness, so a brief refusal is better than a wrong answer.

2. Estimate

Assume 5 million payments a day, about 60 a second on average and 300 at peak. Each payment writes a handful of ledger entries, so around 1,500 writes a second at peak. This is a modest load for one well-run relational database with replicas, which is why a transactional database is the right default here.

3. Double-entry ledger

Every movement of money is a set of entries that sum to zero: a debit on one account and a credit on another. Entries are never updated or deleted. A correction is a new entry that reverses the old one. Balances are derived from entries or kept in a table that is updated in the same transaction. This makes the history auditable and errors visible.

4. Idempotency

The client sends an idempotency key with each request. The service stores the key with the result in the same transaction as the ledger entries. A retry with the same key returns the stored result and does nothing new. The same key with different details is rejected. Keys expire after a window such as 24 hours.

5. State machine and external calls

A payment moves through states: created, authorised, captured, refunded, failed. Only valid transitions are allowed, enforced by the database. A call to the processor can time out with unknown outcome. Record the attempt before calling, use the processor's own idempotency key, and resolve unknown outcomes by querying the processor or by reconciliation, never by guessing.

MARGA by EknathaLabs
marga.eknathalabs.com

6. Failure modes

Failure	Effect	Answer
Client retries	duplicate charge risk	idempotency key
Crash mid-payment	half-done work	single transaction, state machine
Processor timeout	unknown outcome	query status, reconcile
Database failover	brief errors	synchronous replica, retries with same key
Bug writes bad entries	wrong balances	append-only reversal, daily balance checks
Processor and ledger disagree	money mismatch	daily reconciliation with alerts

7. Trade-offs

A single database gives strong consistency and simple correctness, at the cost of scaling limits that you can postpone by partitioning per merchant. Strict state machines are rigid but safe. Asynchronous settlement improves throughput but widens the window where records disagree. Say what you would do to scale past one database.

Two entries that balance

```
$ psql ledger -c "select entry_id, account, debit, credit from entries where payment_id='pay_81f'  \
    order by entry_id"
 entry_id | account         | debit | credit
----------+-----------------+-------+--------
 9001     | customer:441    |  50.00 |
 9002     | merchant:77     |        |  48.50
 9003     | fees:platform   |        |   1.50
$ psql ledger -c "select sum(debit)-sum(credit) from entries where payment_id='pay_81f'"
 0.00
```

Debits equal credits, so the payment balances. A check that every payment sums to zero catches many bugs early. The output is a simulated illustration.

How it fails

A retry charges the customer twice. A crash leaves money debited and not credited. An unknown processor result is guessed. A bug edits history. Reconciliation is skipped and mismatches grow.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: a payments ledger fits together

Part	What it does	What goes wrong
Clarify	Customers pay merchants. The service creates a payment, authorises it with an external processor, captures it, and later refunds some or all of it.	No reconciliation with the processor.
Estimate	Assume 5 million payments a day, about 60 a second on average and 300 at peak.	Updating or deleting ledger rows.
Idempotency	The client sends an idempotency key with each request.	New reference on every retry.
State machine and external calls	A payment moves through states: created, authorised, captured, refunded, failed.	Record the attempt before calling, use the processor's own idempotency key, and resolve unknown outcomes by querying the processor or by...
Trade-offs	A single database gives strong consistency and simple correctness, at the cost of scaling limits that you can postpone by partitioning per merchant.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Two entries that balance	Debits equal credits, so the payment balances.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clarify to Estimate / Estimate to Idempotency / Idempotency to State machine and external calls / State machine and external calls to Trade-offs / Trade-offs to Two entries that balance

Incident: The timeout that charged twice

> **Incident:** *A payment request to an external processor timed out. The service treated the timeout as a failure and retried with a new reference, but the first request had in fact succeeded. The customer was charged twice.*

```
$ ledgerctl payment pay_81f
state: failed (processor timeout)
attempts: 2 (processor refs: A-1032, A-1033)
$ ledgerctl reconcile --date 2026-03-14
processor captured 2, ledger captured 1: MISMATCH
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Using one idempotency key for all attempts, and resolving an unknown outcome by asking the processor, would have prevented the second charge. Daily reconciliation caught the mismatch quickly.

Interview question

"Design a payments ledger." Use double-entry append-only records, idempotency keys, transactions with a state machine, treat timeouts as unknown, reconcile daily, and explain how you would scale a single database later.

Common mistakes

- Updating or deleting ledger rows
- Treating a timeout as a failure
- New reference on every retry
- No reconciliation with the processor

Check yourself

What is the purpose of double-entry records?

Answer: Every movement balances, which makes errors visible and history auditable

How should a timeout from the processor be handled?

MARGA by Ekn... marga.eknathalabs.com

Answer: As an unknown outcome, resolved by querying the processor or reconciling

What does an idempotency key prevent?

Answer: A retried request from creating a second payment

How do you correct a wrong ledger entry?

Answer: Add a reversing entry and a correct one, keeping history

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 33. Worked design: search autocomplete

17 minutes. Suggest completions in a few milliseconds: prefix indexes, ranking, freshness and how to degrade gracefully under load.

Autocomplete is a latency problem. A person types a letter and expects suggestions before the next keystroke, so the budget is tens of milliseconds. The data structure is simple, the interesting parts are ranking suggestions, keeping them fresh, and surviving a burst of traffic, since every keystroke is a request.

What you will learn

- Size a request-per-keystroke service
- Choose a prefix index and a cache layer
- Rank and refresh suggestions
- Degrade safely when load or dependencies fail

1. Clarify

The user types into a search box and sees up to ten suggestions after each keystroke. Suggestions come from popular past queries, filtered for safety. Latency must stay under 50 milliseconds at the 99th percentile. Suggestions may be minutes old, but a failure must never block the search box itself.

2. Estimate

Assume 200 million searches a day with six keystrokes on average that trigger suggestions, so 1.2 billion requests a day, about 14,000 a second and 40,000 at peak. The index of the top ten million queries at 50 bytes each is about 500 MB, small enough to hold in memory on every serving node.

3. Prefix index

Store queries in a trie or a sorted structure, and for each prefix keep the top ten suggestions precomputed. A lookup is then a single map read. Limit prefix length, for example the first 20 characters, and normalise case and accents so near-identical typings share entries.

4. Caching and the client

Most keystroke sequences repeat, so cache by prefix at the edge for a short time. The client waits a small delay after typing, called debouncing, cancels outdated requests, and caches earlier results. Together these can cut the load by more than half and reduce wasted work.

5. Ranking and freshness

Rank suggestions by recent popularity, with a decay so old trends fade, and personalise lightly if the user allows it. Build the index offline from query logs every few minutes to hours, then ship it to serving nodes as a versioned file. A separate fast path can inject trending queries quickly, with an expiry.

MARGA by EknathaLabs
marga.eknathalabs.com

6. Failure modes

Failure	Effect	Answer
Serving node dies	fewer replicas	load balancer health checks, many replicas
Bad index build	wrong suggestions	validate size and samples, staged rollout, rollback
Traffic spike	latency rises	shed load, serve from cache, drop personalisation
Offensive suggestion	reputation harm	blocklist filter at build time and at serve time
Log pipeline lags	stale suggestions	alert on index age, keep last good index
Backend down	no suggestions	the search box still works, show nothing

7. Trade-offs

Precomputing the top ten per prefix costs memory and rebuild time, and buys speed. Fresher indexes cost more compute. Personalisation helps relevance but breaks sharing of cache entries between users. Choose the simplest design that meets the latency target, and say what you would change if the index grew a hundred times.

Checking an index before rollout

```
$ acindex validate --file index-2026-03-14T10.bin
entries: 9,874,102  (previous 9,861,550)  size change +0.1%
sample prefixes checked: 500, empty results: 3
blocklist hits: 0
$ acindex rollout --file index-2026-03-14T10.bin --stage 5%
```

A new index is checked for size, sample coverage and blocked terms before a small staged rollout. If the check fails, the last good index stays in place. The output is a simulated illustration.

How it fails

A bad index build ships to every node. A traffic spike at a big event overloads serving. An offensive suggestion appears. Debouncing is missing, so every keystroke is a request. The index is hours old and nobody notices.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Worked design: search autocomplete fits together

Part	What it does	What goes wrong
Clarify	The user types into a search box and sees up to ten suggestions after each keystroke.	Requests on every keystroke with no debouncing.
Estimate	Assume 200 million searches a day with six keystrokes on average that trigger suggestions, so 1.2 billion requests a day, about 14,000 a second and 40,000 at peak.	Shipping a new index to every node at once.
Caching and the client	Most keystroke sequences repeat, so cache by prefix at the edge for a short time.	No blocklist at serving time.
Ranking and freshness	Rank suggestions by recent popularity, with a decay so old trends fade, and personalise lightly if the user allows it.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Trade-offs	Precomputing the top ten per prefix costs memory and rebuild time, and buys speed.	Personalisation helps relevance but breaks sharing of cache entries between users.
Checking an index before rollout	A new index is checked for size, sample coverage and blocked terms before a small staged rollout.	If the check fails, the last good index stays in place.

Flow: Clarify to Estimate / Estimate to Caching and the client / Caching and the client to Ranking and freshness / Ranking and freshness to Trade-offs / Trade-offs to Checking an index before rollout

Incident: A suggestion nobody should have seen

> **Incident:** *A new index included a phrase that appeared in a burst of spam queries. It showed up as a suggestion for a common word, and users saw it for several hours before the next build.*

```
$ acindex lookup "how to"
1 how to tie a tie
2 how to <redacted spam phrase>
$ acindex blocklist check --index current
hits: 1
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A blocklist filter at build time and at serving time, plus a fast way to remove one entry without a full rebuild, limits how long a bad suggestion can live.

Interview question

"Design search autocomplete." Estimate requests per keystroke, hold a prefix to top-ten map in memory, cache at the edge with client debouncing, rebuild offline with staged rollout, add a blocklist, and shed load under spikes.

Common mistakes

- Requests on every keystroke with no debouncing
- Shipping a new index to every node at once
- No blocklist at serving time
- Letting autocomplete failures break the search box

Check yourself

Why precompute the top suggestions for each prefix?

Answer: A lookup becomes one map read, which meets the latency target

What does debouncing do on the client?

Answer: Waits briefly after typing before sending a request

How do you limit the damage of a bad index build?

Answer: Validate it and roll it out in stages with the last good index kept for rollback

MARGA by EknathaLabs marga.eknathalabs.com

What should happen if the suggestion service is down?

Answer: The search box still works, with no suggestions shown

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: System design for platform and SRE

The design order, estimation numbers, building-block rules and failure-containment patterns on one page.

The order

```
1. Clarify    - users, what must never be wrong, SLO
2. Estimate   - requests per second, storage, bandwidth, peak
3. Sketch     - smallest design that meets the numbers
4. Deep-dive  - the risky part: data store, queue, rollout
5. Failures   - slow, down, wrong, overloaded, for every box
6. Trade-offs - say what you gave up
```

Numbers to remember

```
seconds per day     ~ 100,000 (86,400)
1M requests/day     ~ 10 per second
100M requests/day   ~ 1,000 per second
peak versus average ~ 3x unless known
memory read ~100 ns | SSD read ~100 us | same-region RTT ~1 ms | cross-continent RTT 100-200 ms
99.9% = 43 min/month | 99.99% = 4.3 min/month | 99.999% = 26 s/month
```

Building blocks in one line each

- **Cache:** jitter TTLs, coalesce refills, serve stale, warm before cutover, size the store for the cold case.
- **Queue:** assume at-least-once, make consumers idempotent, bound the queue, watch age not length.
- **Retry:** timeout shorter than the caller, exponential backoff with jitter, retry at one layer, use a budget.
- **Idempotency:** client key, stored with the result in one transaction.
- **Consistency:** pick per feature; fix stale reads with read-your-writes; alert on replication lag.
- **Quorum:** majority of an odd number; 3 tolerates 1, 5 tolerates 2; spread across zones.
- **Fencing:** the resource rejects a lower token; a lease alone is not enough.
- **Time:** monotonic for durations; never order by wall-clock across machines; handle negative differences.

Containing failure

```
failure domain   what one cause takes down together (process, host, zone, region, change)
cell             full independent copy for a slice of users, thin router, hard size limit
bulkhead         separate pools per dependency
shuffle shard    random worker subset per customer
canary           smallest stage with a stop signal and bake time
fail static      keep last known good on bad input
fail open/closed decide per check, write it down, test it
```

Overload

```
token bucket         steady refill, bounded burst
concurrency limit    cap in-flight work (protects slow paths)
shed by priority     batch first, critical last
signals              429 or 503 with Retry-After; clients back off with jitter
autoscaling          slow; limits hold the line meanwhile
```

Multi-region

```
RTO = time down       RPO = data lost, in time
backup/restore < pilot light < warm standby < active-active
survivors must carry the lost share (N regions need headroom)
two DNS providers; drill the failover; recovery tools off the failing system
```

Retry arithmetic

```
layers retrying  3 x 3 x 3  =  27 calls per user request at the bottom
fix: retry at one layer, jittered backoff, budget of ~10% of traffic
```

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Retries without jitter arrive in waves and keep a recovering service down.*

Failover checks

```
# who is writable? exactly one should answer false for recovery
psql -h NODE -Atc "select pg_is_in_recovery()"
# replication lag in seconds on a replica
psql -Atc "select extract(epoch from now()-pg_last_xact_replay_timestamp())"
```

Warning: *Never promote a node without fencing the old primary first, or two writers will diverge.*

Data

```
partition key   many values, even load, keeps related data together; check the biggest value
range           cheap scans, hot end for sequential keys
hash            even load, scans hit all partitions
consistent hash ring + virtual nodes: adding a node moves ~1/N keys
storage engines B-tree: in-place pages, watch vacuum and bloat | LSM: append and compact, watch co \
    mpaction debt
replication     leader/follower | multi-leader (conflicts) | leaderless (R + W > N)
backup          RPO sets frequency, RTO sets restore speed; independent copy; timed restore test
```

Warning: *Replicas copy deletes. A backup you have not restored is a hope.*

By design: observability, security, cost

```
observability  SLI at the edge | golden signals | per-version and per-cell labels | trace id in lo \
    gs
               independent probe and paging path | dead-man's switch | cap cardinality
security       trust boundaries | least privilege | short-lived workload credentials | default-den \
    y network
               separate accounts and keys | audit logs | secret rotation runbook
cost/capacity  measure unit throughput at the SLO | N+1 and zone loss (3 zones: run under ~66%)
               list non-CPU limits | scale on leading signals | write down savings that cut redund \
    ancy
```

Change safety checklist

1. Can it be previewed on real data?
2. Smallest first stage and stop signal?
3. Previous good state and rollback time?
4. What does the consumer do with garbage?
5. Does the fixer depend on the broken thing?

Warning: *A second zone does not protect against a change pushed to every zone at once. Count shared things, not copies.*

Warning: *A status page, runbook or monitoring tool hosted on the failing platform will fail with it. Host them independently.*

Warning: *Automated quotas and limits need a floor. A limit that can reach zero is an outage waiting for a bad input.*

MARGA by EknathaLabs
marga.eknathalabs.com